+ AI

GAME DEVELOPMENT / AI

Add AI to Your

Godot 4.6 Game

The Complete AI Implementation Guide

WHAT'S INSIDE

- Gemini 3 Flash + GLM-5 integration
- Full bridge server & Godot plugin
- MCP — AI controls your editor
- Offline mode on consumer GPU
- 20 secret creative use cases
- Production deployment guide
- Multiplayer AI architecture

Godot 4.6 | Gemini 3 | GLM-5 | MCP

AEiOU

github.com/aeiouofficial

1ST EDITION 2026

63 pages · 25 sections · Complete Edition

Table of Contents

01	Architecture Deep Dive	Component map, data flows, technology choices
02	Complete Setup	Godot 4.6, Python 3.12, API keys, project structure
03	Gemini 3 Flash	Text, vision, function calling, JSON mode, thinking levels
04	GLM-5	Preserved Thinking, tool calling, OpenRouter, vLLM deploy
05	Provider Router	Cost-aware fallback chain with circuit breakers
06	Bridge Server	FastAPI, auth, rate limiting, logging, all endpoints
07	Godot 4.6 Plugin	Autoload singleton, editor plugin, Unique Node IDs
08	Scene Builder	Generation, rollback / checkpoint, UID tracking
09	MCP -- All Six Repos	Runtime control, editor automation, dual-server config
10	Offline Mode	Ollama + GLM-5 via vLLM / SGLang
11	Streaming	SSE, GDScript HTTPClient, typewriter effect
12	Error Handling & Retry	Per-provider taxonomy, backoff, JSON repair
13	Context Caching	90% cost reduction, implicit vs explicit caching
14	GLM-5 Multi-Turn Agent	Preserved Thinking, stateful autonomous session
15	Testing	pytest mocks, contract tests, Godot headless CI
16	CI/CD & Docker	Full pipeline, systemd, nginx, secrets management
17	Model & Cost Reference	Live pricing, benchmarks, decision matrix
18	Standard Use Cases	NPC dialogue, quest gen, difficulty, voice commands
19	Advanced Use Cases	Autonomous playtester, economy balancer, localisation
20	Performance Optimisation	Token compression, batching, parallel provider racing
21	Cost Tracking & Guardrails	Per-session spend, hard caps, structured logging
22	Production Deployment	systemd, nginx TLS, HMAC signing, crash recovery
23	Multiplayer AI	Server authority, shared NPC state, anti-cheat
24	Godot 4.6 Advanced Patterns	LibGodot, IKModifier3D, ObjectDB snapshot debug
25	Secret Creative Use Cases	Twenty techniques most developers have never tried
APP	Reference Appendix	All endpoints, env vars, error codes, glossary

§ 01

Architecture Deep Dive

Understand every component before touching a line of code. Know what you are building and why.

Why a separate Python server?

The first question most Godot developers ask is: why not call the AI API directly from GDScript? The answer is authentication security, flexibility, and capability. GDScript has no way to safely store API keys — any HTTP call from Godot ships your key in plain text inside the binary. A local Python server keeps keys in a .env file, adds retry logic, manages multiple providers, handles streaming, and can run VLM screenshot analysis — none of which GDScript can do alone. The server runs on localhost:8000. Your game never talks to OpenAI or Google directly.

The complete architecture

```
■■■
```

Flow	Trigger	Path	Result
Offline fallback	Any trigger above	router detects cloud unreachable → Ollama :11434	Same result, local model, no internet

Technology choices explained

Choice	Why not the alternatives
FastAPI over Flask	FastAPI is async-native, typed, auto-generates docs at /docs, 15-30% faster than Flask for concurrent requests. Godot sends multiple AI calls.
HTTPRequest for non-streaming, HTTPClient for streaming	HTTPRequest is simpler but blocks on chunks. HTTPClient lets GDScript read partial response bodies chunk-by-chunk — essential for live token display.
Gemini 3 Flash as primary over GLM-5	1M context vs 200K. Vision built-in (needed for VLM loop). Implicit context caching auto-saves 90% on repeated scene context.
GLM-5 as fallback over GPT-4o	MIT license — can self-host. OpenAI-SDK compatible, drop-in swap. 77.8% SWE-bench, comparable quality. Preserved Thinking across turns.
Ollama over llama.cpp direct	Ollama manages model downloads, quantisation, VRAM, and serves an OpenAI-compatible REST API on :11434. One command install.

§ 02

Complete Setup

Every command, every file, every troubleshoot — from nothing to a running bridge server.

■ GODOT 4.6 — THREE CHANGES THAT AFFECT YOUR AI CODE

(1) Unique Node IDs: get_instance_id() now returns a stable ID that survives rename and reparenting. Track AI-created nodes by ID, not path. (2) LibGodot: Godot can run embedded — useful for headless screenshot capture in CI without a display server. (3) Jolt Physics default: AI-generated RigidBody3D and CharacterBody3D nodes use Jolt. If you use physics properties in AI-generated scenes, test Jolt behaviour — it differs from Bullet/GodotPhysics in edge cases (tunnelling, constraints, sleep thresholds).

2.1 Install Godot 4.6

Download from godotengine.org/releases/4.6. Get the standard build unless you need C# (Mono) support. On Linux, make the binary executable: chmod +x Godot_v4.6-stable_linux.x86_64. Optionally add it to PATH so you can launch it from the terminal for headless operations.

2.2 Python environment

BASH terminal

```
# Verify Python 3.12+ python3 --version # need 3.12.x or higher # Create the bridge folder inside your Godot project cd /path/to/your-godot-game mkdir ai_bridge && cd ai_bridge # Virtual environment (keeps packages isolated from system Python) python3.12 -m venv .venv source .venv/bin/activate # Windows: .venv\Scripts\activate # Core dependencies pip install fastapi "uvicorn[standard]" httpx python-dotenv # AI provider SDKs pip install google-genai openai # VLM / image support pip install pillow # Testing (optional but recommended) pip install pytest pytest-asyncio # Save requirements pip freeze > requirements.txt
```

2.3 API Keys

■ NEVER PUT KEYS IN GODOT OR GIT

Your GDScript code must never contain an API key. Keys in Godot scripts are embedded in the exported binary and trivially extractable. Keys in git history are compromised forever even after deletion. Keys live ONLY in ai_bridge/.env, which must be in .gitignore.

BASH ai_bridge/.env (create this file, never commit it)

```
# Google Gemini 3 Flash — get from aistudio.google.com GEMINI_API_KEY=AIzaSy_your_key_here # z.ai for GLM-5 — get from z.ai developer portal ZAI_API_KEY=your_z_ai_key_here # Ollama (optional — for offline mode) OLLAMA_URL=http://localhost:11434 OLLAMA_MODEL=glm4 # Set to true to skip all cloud calls (useful for CI, offline dev) AI_OFFLINE=false # Bridge server settings BRIDGE_PORT=8000 BRIDGE_HOST=127.0.0.1 LOG_LEVEL=INFO
```

BASH .gitignore (add these lines)

```
# AI bridge secrets and caches ai_bridge/.env ai_bridge/.venv/ ai_bridge/__pycache__/ ai_bridge/*.pyc
/tmp/godot_screenshot.png
```

2.4 Project folder structure

```
your-game/ ■■■ project.godot ■■■ .gitignore ← add .env and .venv here ■■■ .mcp.json ← MCP config for
Claude Code (§09) ■ ■■■ addons/ai_agent/ ← Godot plugin (§07) ■ ■■■ plugin.cfg ■ ■■■
ai_agent_plugin.gd ← EditorPlugin — registers autoload ■ ■■■ ai_agent.gd ← Main autoload node (call
from anywhere) ■ ■■■ scene_builder.gd ← Applies AI-generated scene commands (§08) ■ ■■■ env_patcher.gd
← WorldEnvironment AI tuning (§10) ■ ■■■ ai_streamer.gd ← Streaming token-by-token display (§11) ■ ■■■
ai_bridge/ ← Python server (runs separately) ■■■ .env ← SECRET: API keys ■■■ requirements.txt ■■■
bridge_server.py ← FastAPI REST server (§06) ■■■ provider_router.py ← Gemini → GLM-5 → Ollama chain
(§05) ■■■ provider_gemini.py ← Gemini 3 Flash client (§03) ■■■ provider_glm.py ← GLM-5 client (§04)
■■■ offline_handler.py ← Ollama client (§10) ■■■ vlm_feedback.py ← VLM screenshot loop ■■■
glm_agent.py ← Multi-turn stateful GLM-5 agent ■■■ cache_manager.py ← Gemini explicit context caching
■■■ token_budget.py ← Cost guardrails ■■■ tests/ ■■■ conftest.py ← Mock fixtures ■■■ test_router.py
■■■ test_bridge.py
```

2.5 Troubleshooting common setup problems

Problem	Cause	Fix
"No module named google.genai"	Wrong package name	pip install google-genai (not google-generativeai)
"GEMINI_API_KEY not found"	.env not loaded	Add from dotenv import load_dotenv; load_dotenv() at top of each provider file
"Connection refused on :8000" from Godot	Bridge not running or wrong host	Start python bridge_server.py first. Check BRIDGE_HOST=127.0.0.1 not 0.0.0.0
"Request timed out" on first call	Gemini cold start or thinking enabled	Use thinking="none" for first smoke test. thinking="high" can take 15-25s.
Godot scene changes not persisting	owner not set on AI-created nodes	Always set node.owner = get_tree().edited_scene_root after add_child()
MCP says "Godot is offline"	Addon not enabled or WebSocket port blocked	Project Settings → Plugins → Godot MCP → Enable. Check firewall on port 6550.
venv not activating on Windows	Execution policy	Run: Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass

§ 03

Gemini 3 Flash — Every Feature

The fastest, cheapest model that still beats GPT-4-class models on coding. Use it first for everything.

■ MODEL FACTS — March 15, 2026

Model string: gemini-3-flash-preview Context: 1,000,000 tokens. Input: $0.50/M · Output: $3.00/M · Cache: $0.05/M (90% off input). SWE-bench Verified: 78.0%. Throughput: 380+ t/s. Vision: yes (images, video). Function calling: yes (parallel). JSON mode: yes. Streaming: yes. thinking_level: none / minimal / medium / high. DEPRECATED predecessor: gemini-2.0-flash — shuts down June 1 2026.

3.1 Complete provider file

PYTHON ai_bridge/provider_gemini.py

```
from google import genai from google.genai import types import asyncio, os, base64, pathlib from dotenv
import load_dotenv load_dotenv() client = genai.Client(api_key=os.environ["GEMINI_API_KEY"]) MODEL =
"gemini-3-flash-preview" # Thinking budgets — maps semantic name → token budget # "none" = no
chain-of-thought, fastest response # "high" = up to 32K reasoning tokens, best quality THINK = { "none":
types.ThinkingConfig(thinking_budget=0), "minimal": types.ThinkingConfig(thinking_budget=512), "medium":
types.ThinkingConfig(thinking_budget=8192), "high": types.ThinkingConfig(thinking_budget=32768), }
```

PYTHON ai_bridge/provider_gemini.py (continued)

```
async def gemini_complete( prompt: str, system: str = "", thinking: str = "medium", image_path: str|None
= None, json_schema: dict|None = None, # force JSON output ) -> dict: contents: list = [] if image_path: #
VLM: pass image as PIL or bytes try: import PIL.Image contents.append(PIL.Image.open(image_path)) except
Exception: # Fallback: base64 inline data data = pathlib.Path(image_path).read_bytes() b64 =
base64.b64encode(data).decode() contents.append({"inline_data": { "mime_type": "image/png", "data":
b64}}) contents.append(prompt) cfg_kwargs: dict = { "system_instruction": system or "You are a Godot 4.6
expert.", "temperature": 0.2, "max_output_tokens": 8192, "thinking_config": THINK[thinking], } if
json_schema: cfg_kwargs["response_mime_type"] = "application/json" cfg_kwargs["response_schema"] =
json_schema resp = await asyncio.to_thread( client.models.generate_content, model=MODEL,
contents=contents, config=types.GenerateContentConfig(**cfg_kwargs), ) u = resp.usage_metadata return {
"text": resp.text, "tokens": u.total_token_count, "cached": getattr(u,"cached_content_token_count",0),
"think_t": getattr(u,"thoughts_token_count",0), "model": MODEL, }
```

3.2 Function calling (Gemini tools)

Function calling lets Gemini return structured actions instead of free text. This is how your AI tells your game engine to DO things — create nodes, set properties, run animations — with type-safe parameters that you validate before executing.

PYTHON ai_bridge/provider_gemini.py (tool calling)

```
from google.genai import types as gt # Define tools Gemini can call — mirrors your Godot capabilities
GODOT_TOOLS = [gt.Tool(function_declarations=[ gt.FunctionDeclaration( name="create_node",
description="Create a node in the Godot scene tree.", parameters=gt.Schema(type=gt.Type.OBJECT,
properties={ "type": gt.Schema(type=gt.Type.STRING, description="Godot class: MeshInstance3D, CSGBox3D,
etc."), "name": gt.Schema(type=gt.Type.STRING), "parent": gt.Schema(type=gt.Type.STRING,
description="Node path, default ."), }, required=["type","name"]), ), gt.FunctionDeclaration(
name="set_property", description="Set a property on an existing Godot node.",
parameters=gt.Schema(type=gt.Type.OBJECT, properties={ "path": gt.Schema(type=gt.Type.STRING),
"property": gt.Schema(type=gt.Type.STRING), "value": gt.Schema(type=gt.Type.STRING), },
required=["path","property","value"]), ), gt.FunctionDeclaration( name="run_gdscript",
description="Execute a GDScript snippet in the running game.", parameters=gt.Schema(type=gt.Type.OBJECT,
properties={ "code": gt.Schema(type=gt.Type.STRING, description="Valid GDScript 2.0 code"), },
required=["code"]), ), ])] async def gemini_agent(prompt: str) -> dict: """Use Gemini with tools —
returns either a tool call or free text."""  model = genai.GenerativeModel(MODEL, tools=GODOT_TOOLS) chat
= model.start_chat() resp = chat.send_message(prompt) for part in resp.candidates[0].content.parts: if
hasattr(part,"function_call") and part.function_call: fc = part.function_call return
{"action":"tool","name":fc.name,"args":dict(fc.args)} return {"action":"text","text":resp.text}
```

3.3 JSON mode — structured output every time

JSON mode guarantees Gemini returns valid JSON matching a schema. Essential for scene builder commands, environment patches, and any AI output you parse programmatically. Pass a response_schema and set response_mime_type.

PYTHON usage example — force JSON output

```
# Schema for scene builder commands SCENE_CMD_SCHEMA = { "type": "array", "items": { "type": "object",
"properties": { "action": {"type":"string","enum":["create","set","delete"]}, "type": {"type":"string"},
"name": {"type":"string"}, "parent": {"type":"string"}, "path": {"type":"string"}, "property":
{"type":"string"}, "value": {"type":"string"}, }, "required": ["action"], } } # Now Gemini WILL return
valid JSON matching this schema result = await gemini_complete( prompt="Build a campfire: fire particles,
logs, warm glow light.", thinking="medium", json_schema=SCENE_CMD_SCHEMA, ) # result["text"] is
guaranteed valid JSON — no parsing guards needed import json commands = json.loads(result["text"])
```

3.4 thinking_level guide for game developers

thinking_level	Budget	Approx TTFT	Cost add	Use this for
"none"	0 tokens	<1 second	$0	Property gets, simple lookups, is-this-valid checks
"minimal"	512 tokens	~1 second	+$0.0015	NPC one-liner dialogue, yes/no decisions
"medium"	8192 tokens	~6 seconds	+$0.024	GDScript generation, scene layout, default for most tasks
"high"	32768 tokens	~15 seconds	+$0.098	Architecture design, complex debug, multi-file refactor

■ RULE OF THUMB

Start every new task with thinking="medium". If quality is fine, downgrade to "minimal" or "none" to save cost and latency. Only use "high" for tasks where you genuinely need the AI to think through multiple competing approaches — like designing a boss AI behavior tree.

§ 04

GLM-5 — Full Integration

MIT-licensed, 744B/40B MoE, Preserved Thinking, tool calling. The best open-source model available.

■ MODEL FACTS — February 11, 2026 release

Model string (z.ai API): glm-5 Parameters: 744B total, 40B active per token (DeepSeek Sparse Attention MoE). Context: 200K tokens, 128K max output. SWE-bench Verified: 77.8%. LMArena Text Arena: rank 11 overall, #1 open-source. Input: $1.00/M · Output: $3.20/M (z.ai). OpenRouter: $0.72/$2.30/M. MIT licensed — weights on Hugging Face at zai-org/GLM-5. Inference: 63.5 t/s median (Artificial Analysis). Three thinking modes: Interleaved, Preserved (multi-turn), Turn-level toggle.

4.1 Complete provider file

PYTHON — ai_bridge/provider_glm.py

```
from openai import AsyncOpenAI import os, json from dotenv import load_dotenv load_dotenv() # GLM-5 is fully OpenAI-SDK compatible client = AsyncOpenAI( api_key = os.environ["ZAI_API_KEY"], base_url = "https://api.z.ai/api/paas/v4", ) MODEL = "glm-5" async def glm_complete( prompt: str, system: str = "", thinking: bool = True, tools: list|None = None, history: list|None = None, # for multi-turn agents ) -> dict: """ thinking=True → Preserved Thinking enabled (clear_thinking=False) history → pass previous messages for multi-turn sessions """ if history: msgs = [{"role":"system","content":system or "You are a Godot 4.6 expert."}] msgs.extend(history) msgs.append({"role":"user","content":prompt}) else: msgs = [ {"role":"system","content":system or "You are a Godot 4.6 expert."}, {"role":"user","content":prompt}, ] extra: dict = {} if thinking: extra["thinking"] = {"type": "enabled"} extra["clear_thinking"] = False # Preserved Thinking — retain across turns kw = dict(model=MODEL, messages=msgs, temperature=0.2, max_tokens=8192, **extra) if tools: kw["tools"] = tools; kw["tool_choice"] = "auto" resp = await client.chat.completions.create(**kw) msg = resp.choices[0].message if msg.tool_calls: tc = msg.tool_calls[0] return { "tool": tc.function.name, "args": json.loads(tc.function.arguments), "thinking": getattr(msg,"reasoning_content",None), "model": MODEL, } return { "text": msg.content, "thinking": getattr(msg,"reasoning_content",None), "tokens": resp.usage.total_tokens, "model": MODEL, }
```

4.2 Three thinking modes in practice

PYTHON — ai_bridge/provider_glm.py (thinking mode examples)

```
# MODE 1: Interleaved Thinking (default when thinking=True) # Model reasons before EVERY tool call and response. # Good for: complex debugging, architecture decisions. # MODE 2: Preserved Thinking (clear_thinking=False) # Reasoning blocks persist across ALL conversation turns. # Model remembers WHY it made previous decisions. # Essential for: multi-turn level building, long debug sessions. # MODE 3: Turn-level toggle (per-message control) # Disable for fast turns (property lookups), enable for complex turns. async def glm_turn_level( msgs: list, complex_task: bool = True, ) -> dict: """Per-turn thinking control — optimize latency without losing quality."""  resp = await client.chat.completions.create( model=MODEL, messages=msgs, extra_body={ "thinking": { "type": "enabled" if complex_task else "disabled" }, "clear_thinking": False, # always preserve across turns }, max_tokens=4096, ) msg = resp.choices[0].message return { "text": msg.content, "thinking": getattr(msg,"reasoning_content",None), "tokens": resp.usage.total_tokens, } # Usage in an agent loop: # Turn 1 (design the level): complex_task=True → ~8s, full reasoning # Turn 2 (set fog_density): complex_task=False → ~1s, no overhead # Turn 3 (fix a script bug): complex_task=True → ~8s, full reasoning
```

4.3 GLM-5 on OpenRouter (cheaper, multi-provider routing)

OpenRouter gives you GLM-5 at $0.72/$2.30/M — 28% cheaper than z.ai — plus automatic failover to other providers if z.ai is down. The client code is identical, just change the base_url.

PYTHON ai_bridge/provider_glm.py (OpenRouter alternative)

```
# To use OpenRouter instead of z.ai: # 1. Get API key from openrouter.ai # 2. Add OPENROUTER_API_KEY to
.env # 3. Swap the client: import os USE_OPENROUTER = os.getenv("USE_OPENROUTER","false").lower() ==
"true" if USE_OPENROUTER: client = AsyncOpenAI( api_key = os.environ["OPENROUTER_API_KEY"], base_url =
"https://openrouter.ai/api/v1", ) MODEL = "zai-org/glm-5" # OpenRouter model identifier else: client =
AsyncOpenAI( api_key = os.environ["ZAI_API_KEY"], base_url = "https://api.z.ai/api/paas/v4", ) MODEL =
"glm-5"
```

4.4 GLM-5 tools — Godot-specific definitions

PYTHON ai_bridge/provider_glm.py (Godot tools)

```
# Standard set of tools GLM-5 can call to control Godot. # Mirror these on the Godot side in
scene_builder.gd. GODOT_TOOLS = [ {"type":"function","function":{"name":"create_node",
"description":"Create a node in the Godot scene tree.",
"parameters":{"type":"object","required":["type","name"], "properties":{ "type":
{"type":"string","description":"Godot class name"}, "name": {"type":"string"}, "parent":
{"type":"string","default":"."}, "props": {"type":"object","description":"Initial properties"}, }}}},
{"type":"function","function":{"name":"set_property", "description":"Set any property on an existing
node.", "parameters":{"type":"object","required":["path","property","value"], "properties":{ "path":
{"type":"string"}, "property": {"type":"string"}, "value": {"type":"string"}, }}}},
{"type":"function","function":{"name":"attach_script", "description":"Write and attach a GDScript to a
node.", "parameters":{"type":"object","required":["path","code"], "properties":{ "path":
{"type":"string","description":"Node path"}, "code": {"type":"string","description":"GDScript 2.0
source"}, "fname": {"type":"string","description":"Script filename"}, }}}},
{"type":"function","function":{"name":"delete_node", "description":"Remove a node from the scene.",
"parameters":{"type":"object","required":["path"], "properties":{"path":{"type":"string"}}}}}, ]
```

§ 05

Production Provider Router

Cost-aware, latency-aware, task-aware routing with circuit breakers and observability.

■ ROUTING STRATEGY

The basic router tries providers in order: Gemini → GLM-5 → Ollama. The production router is smarter: short prompts (<2K tokens) skip GLM-5 and go Gemini → Ollama (GLM-5 is overkill and slower at $1/M). Vision tasks always use Gemini (GLM-5 is text-only). Long context (>200K) uses Gemini 3.1 Pro (1M ctx). Cost-sensitive CI builds always use Ollama.

PYTHON — ai_bridge/provider_router.py

```
import asyncio, httpx, logging, time, os from provider_gemini import gemini_complete from provider_glm
import glm_complete from offline_handler import ollama_complete log = logging.getLogger("router") # ■■
Health checks ■■■
```

§ 06

Bridge Server — Full Production

Every endpoint your game will ever need. Logging, rate limiting, auth, docs at /docs.

PYTHON ai_bridge/bridge_server.py

```
from fastapi import FastAPI, HTTPException, Request, Depends from fastapi.responses import
StreamingResponse, JSONResponse from fastapi.middleware.cors import CORSMiddleware from pydantic import
BaseModel, Field from dotenv import load_dotenv from provider_router import route import uvicorn,
logging, os, time load_dotenv() logging.basicConfig(level=os.getenv("LOG_LEVEL","INFO"),
format="%(asctime)s %(name)-12s %(levelname)s %(message)s") log = logging.getLogger("bridge") app =
FastAPI( title="Godot AI Bridge", version="1.0.0", description="AI provider bridge for Godot 4.6", ) #
Allow Godot to call from localhost app.add_middleware(CORSMiddleware,
allow_origins=["http://localhost","http://127.0.0.1"], allow_methods=["*"], allow_headers=["*"]) # Simple
per-IP rate limiting (production: use Redis) _req_times: dict[str, list] = {} def rate_limit(request:
Request, limit: int = 30, window: int = 60): ip = request.client.host now = time.time()
_req_times.setdefault(ip,[]) _req_times[ip] = [t for t in _req_times[ip] if now-t < window] if
len(_req_times[ip]) >= limit: raise HTTPException(429, "Rate limit exceeded") _req_times[ip].append(now)
```

PYTHON ai_bridge/bridge_server.py (endpoints)

```
class AiReq(BaseModel): prompt: str system: str = "" thinking: str = Field("medium",
pattern="^(none|minimal|medium|high)$") provider: str = Field("auto",
pattern="^(auto|gemini|glm-5|ollama)$") class EnvPatch(BaseModel): patch: dict @app.get("/health") async
def health(): return {"ok":True,"version":"1.0.0","ts":time.time()} @app.get("/providers") async def
providers(): """Returns which providers are currently reachable."""  from provider_router import
_ping_gemini,_ping_glm,_ping_ollama return { "gemini": await _ping_gemini(), "glm-5": await _ping_glm(),
"ollama": await _ping_ollama(), } @app.post("/ai/complete") async def complete(req: AiReq,
_=Depends(rate_limit)): log.info(f"complete: provider={req.provider} thinking={req.thinking}") if
req.provider != "auto": fns = {"gemini":"provider_gemini.gemini_complete",
"glm-5":"provider_glm.glm_complete", "ollama":"offline_handler.ollama_complete"} mod,fn =
fns[req.provider].rsplit(".",1) import importlib r = await
getattr(importlib.import_module(mod),fn)(req.prompt,req.system) return
{"provider":req.provider,"result":r} return await
route(req.prompt,system=req.system,thinking=req.thinking) @app.post("/env/patch") async def
env_patch(body: EnvPatch): """VLM loop pushes patches here; Godot polls or WebSocket picks them up."""
log.info(f"env/patch: {body.patch}") return {"status":"queued","patch":body.patch}
@app.post("/scene/screenshot") async def trigger_screenshot(): """Ask Godot to take a screenshot via
MCP."""  return {"status":"screenshot_requested"} @app.get("/metrics") async def metrics(): """Basic
metrics for monitoring."""  from provider_router import _fails return {"circuit_fails": {k:len(v) for k,v
in _fails.items()}} if __name__ == "__main__": host = os.getenv("BRIDGE_HOST","127.0.0.1") port =
int(os.getenv("BRIDGE_PORT","8000")) log.info(f"Starting bridge on {host}:{port}")
uvicorn.run(app,host=host,port=port,reload=True, log_level=os.getenv("LOG_LEVEL","info").lower())
```

■ BROWSE API DOCS

Once the bridge is running, open http://localhost:8000/docs in a browser. FastAPI auto-generates Swagger UI where you can test every endpoint manually. This is the fastest way to debug your provider setup without touching Godot.

§ 07

Godot 4.6 Plugin — Complete

Three files. One autoload. Call AIAgent.ask() from any script in your entire project.

■ INSTALL ORDER

1. Create the folder addons/ai_agent/ in your project root. 2. Create the three files below. 3. Project → Project Settings → Plugins → "AI Agent" → Enable. 4. Godot will add AIAgent as an autoload singleton automatically. 5. Call AIAgent.ask("your prompt") from any .gd file in your project.

INI — addons/ai_agent/plugin.cfg

```
[plugin] name="AI Agent" description="Godot 4.6 AI bridge — Gemini 3 Flash + GLM-5 + Offline"
author="Your Studio" version="1.0.0" script="ai_agent_plugin.gd"
```

GDSCRIPT — addons/ai_agent/ai_agent_plugin.gd

```
@tool extends EditorPlugin func _enter_tree() -> void: add_autoload_singleton("AIAgent",
"res://addons/ai_agent/ai_agent.gd") print("[AI Agent] plugin loaded — AIAgent autoload active") func
_exit_tree() -> void: remove_autoload_singleton("AIAgent")
```

GDSCRIPT — addons/ai_agent/ai_agent.gd

```
extends Node const BRIDGE := "http://127.0.0.1:8000" const TIMEOUT := 90.0 ## Emitted when the AI returns
a response. ## Connect this in any scene that needs AI output. signal response_received(text: String,
provider: String) ## Emitted if the HTTP call or JSON parsing fails. signal request_error(message:
String) ## Internal: pending request counter for UI spinners signal loading_changed(is_loading: bool) var
_pending := 0 ## Main entry point. Call from any script in your game. ## prompt: what to ask the AI ##
thinking: "none"|"minimal"|"medium"|"high" — controls reasoning depth ## provider:
"auto"|"gemini"|"glm-5"|"ollama" — usually leave as auto func ask( prompt: String, thinking: String =
"medium", provider: String = "auto", ) -> void: _set_loading(true) var http := HTTPRequest.new()
add_child(http) http.timeout = TIMEOUT http.request_completed.connect(_on_done.bind(http)) var body :=
JSON.stringify({ "prompt": prompt, "thinking": thinking, "provider": provider, }) var err :=
http.request( BRIDGE + "/ai/complete", ["Content-Type: application/json"], HTTPClient.METHOD_POST, body)
if err != OK: _set_loading(false) request_error.emit("HTTPRequest failed: " + str(err)) ## Convenience:
ask for scene commands (returns JSON array). ## Connect response_received and parse the JSON array
yourself, ## or pass to SceneBuilder.apply(). func ask_scene_commands( instruction: String, provider:
String = "auto", ) -> void: var full := (instruction + "\nReturn ONLY a valid JSON array of scene
commands." + " Each object must have an action field: create, set, or delete." + " No explanation, no
markdown, just the JSON array.") ask(full, "medium", provider) ## Check if the bridge server is reachable
before asking. func ping(callback: Callable) -> void: var http := HTTPRequest.new() add_child(http)
http.timeout = 3.0 http.request_completed.connect(func(r,code,h,b): http.queue_free() callback.call(code
== 200)) http.request(BRIDGE + "/health") func _on_done( _res, code: int, _h, body: PackedByteArray,
http: HTTPRequest ) -> void: http.queue_free() _set_loading(false) if code != 200:
request_error.emit("HTTP error " + str(code)) return var json :=
JSON.parse_string(body.get_string_from_utf8()) if json == null: request_error.emit("Invalid JSON from
bridge") return var result = json.get("result", json) var text = result.get("text","") if result is
Dictionary else str(result) var prov = json.get("provider","?") response_received.emit(text, prov) func
_set_loading(v: bool) -> void: _pending = max(0, _pending + (1 if v else -1))
loading_changed.emit(_pending > 0)
```

Usage examples from any GDScript

GDSCRIPT any scene or script in your game

```
# ■■ Basic usage ■■■
```

§ 08

Scene Builder — Advanced

Multi-step generation, validation, rollback, and Godot 4.6 Unique Node ID tracking.

GDSCRIPT addons/ai_agent/scene_builder.gd

```
extends Node ## Applies AI-generated scene command arrays to the live scene tree. ## Supports rollback:
undo all changes since last checkpoint. # Godot 4.6: track nodes by stable instance ID (survives
rename/reparent) var _id_map: Dictionary = {} # name → instance_id var _snapshots: Array = [] # rollback
stack ## Apply a JSON array of scene commands from the AI. ## Call after parsing AI response — see
ai_agent.gd usage example. func apply(commands: Array) -> Dictionary: """Returns {ok: bool, applied: int,
errors: Array}""" var result := {"ok":true,"applied":0,"errors":[]} for cmd in commands: var err :=
_exec(cmd) if err: result.errors.append(err) result.ok = false else: result.applied += 1 return result ##
Snapshot current scene state for rollback. func checkpoint() -> void:
_snapshots.append(_id_map.duplicate(deep=true)) ## Undo all changes since last checkpoint. func
rollback() -> void: if _snapshots.is_empty(): return var snap: Dictionary = _snapshots.pop_back() #
Remove nodes created after snapshot for name in _id_map.keys(): if name not in snap: var id: int =
_id_map[name] var node := instance_from_id(id) if node: node.queue_free() _id_map = snap func _exec(cmd:
Dictionary) -> String: match cmd.get("action","create"): "create": return _create(cmd) "set": return
_set(cmd) "delete": return _delete(cmd) return "Unknown action: " + cmd.get("action","") func
_create(cmd: Dictionary) -> String: var cls_name: String = cmd.get("type","Node") if not
ClassDB.class_exists(cls_name): return "Unknown class: " + cls_name var node =
ClassDB.instantiate(cls_name) node.name = cmd.get("name","AINode") var parent_path: String =
cmd.get("parent",".") var parent := get_node_or_null(parent_path) if not parent: node.queue_free() return
"Parent not found: " + parent_path parent.add_child(node) # GODOT 4.6: set owner so node persists in
saved scenes node.owner = get_tree().edited_scene_root # Track by Unique Node ID — stable across
rename/reparent _id_map[node.name] = node.get_instance_id() # Apply initial properties for k in
cmd.get("props",{}).keys(): if node.get(k) != null: node.set(k, cmd.props[k]) else:
push_warning("[SceneBuilder] Ignored unknown prop: " + k) return "" func _set(cmd: Dictionary) -> String:
var node := get_node_or_null(cmd.get("path","")) if not node: return "Node not found: " +
cmd.get("path","") var prop: String = cmd.get("property","") if node.get(prop) == null and not
prop.contains("/"): return "Unknown property " + prop + " on " + node.name node.set(prop,
cmd.get("value")) return "" func _delete(cmd: Dictionary) -> String: var node :=
get_node_or_null(cmd.get("path","")) if not node: return "Node not found: " + cmd.get("path","")
_id_map.erase(node.name) node.queue_free() return "" ## Get a previously AI-created node by name using
Unique ID. ## Survives rename and reparenting — unlike get_node("path"). func get_ai_node(name: String)
-> Node: if name not in _id_map: return null return instance_from_id(_id_map[name])
```

§ 09

MCP Deep Dive — All Repos Compared

Every Godot MCP server, what it can do, when to use which, and how to configure it.

■ MCP IS FOR YOUR EDITOR — NOT YOUR RUNNING GAME

MCP (Model Context Protocol) connects AI assistants like Claude Code or Cursor to your Godot editor. When you type "create a player scene" in Claude Code, the MCP server translates that into Godot API calls in real time. This is separate from your in-game AI (which uses the bridge server on port 8000). You can use both simultaneously — bridge for runtime AI, MCP for editor automation.

9.1 MCP server comparison matrix

Repo	Install	Port	Standout Feature	Use if…
github.com/satelliteoflove/godot-mcp	npx one-liner	6550	Runtime control, screenshots, animations	Starting out — easiest setup
github.com/Coding-Solo/godot-mcp	npm + build	auto	UID handling, Godot 4.4+ compatibility	Need UID/GUID support
github.com/tomyud1/godot-mcp	npx godot-mcp-server	6505	Browser visualiser at :6510	Want visual project map
github.com/ee0pdt/Godot-MCP	npm build	stdio	Full script + scene read/write	Need deep script editing
github.com/bradypp/godot-mcp	npm build	auto	Auto Godot detection, read-only mode	CI/read-only analysis
ryanmazzolini/minimal-godot-mcp	npm	LSP	LSP diagnostics, no addon needed	Want lint without installing addon

9.2 Recommended: @satelliteoflove/godot-mcp v2.11.0

REPO https://github.com/satelliteoflove/godot-mcp

BASH terminal — install once per project

```bash
# Install the Godot addon directly into your project: npx @satelliteoflove/godot-mcp --install-addon /path/to/your-game # In Godot editor: # Project → Project Settings → Plugins → "Godot MCP" → Enable # The addon starts a WebSocket server on port 6550. # Verify the MCP server responds: npx -y @satelliteoflove/godot-mcp & echo '{"jsonrpc":"2.0","id":1,"method":"initialize","params":{"protocolVersion":"2024-11-05","capabilities":{},"clientInfo":{"name":"test","version":"0"}}}' | npx -y @satelliteoflove/godot-mcp # Expected: {"result":{"serverInfo":{"name":"godot-mcp","version":"2.11.0"},...}}
```

JSON .mcp.json (Claude Code / Claude Desktop / Cursor)

```
{ "mcpServers": { "godot": { "command": "npx", "args": ["-y", "@satelliteoflove/godot-mcp"], "env": {
"GODOT_HOST": "localhost", "GODOT_PORT": "6550" } } } }
```

9.3 What you can say to Claude Code once MCP is active

Claude Code prompt	What Godot does
"Create a player with CharacterBody3D, CollisionShape3D, MeshInstance3D"	Three nodes created live in scene tree with correct parent-child hierarchy
"Run the project and show me any errors in the console"	Godot launches in debug mode, output captured, errors returned to Claude
"Take a screenshot of the 3D viewport"	Screenshot saved, shown to Claude — it can see your actual scene
"List all nodes in the current scene"	Full scene tree returned as structured text
"Attach a movement script to Player using WASD"	GDScript written and attached to the node
"Set the SpotLight3D energy to 3.5"	Property updated live, visible in editor immediately
"Add a NavMesh and NavigationAgent3D to the enemy"	Navigation setup created and configured
"Find all syntax errors in my scripts"	All scripts scanned, errors reported with file and line number

9.4 Coding-Solo/godot-mcp — for Godot 4.4+ UID support

REPO https://github.com/Coding-Solo/godot-mcp

Godot 4.4 introduced UIDs for all resources. The Coding-Solo server has explicit get_uid and update_project_uids tools — essential if you are working on a project that was upgraded from Godot 4.3 or earlier and has UID mismatch warnings.

JSON .mcp.json (Coding-Solo config)

```
{ "mcpServers": { "godot": { "command": "node", "args": ["/path/to/godot-mcp/build/index.js"], "env": {
"DEBUG": "true" }, "autoApprove": [ "launch_editor","run_project","get_debug_output",
"create_scene","add_node","save_scene","get_uid" ] } } }
```

9.5 tomyud1/godot-mcp — with browser visualizer

REPO https://github.com/tomyud1/godot-mcp

Unique feature: exposes a web UI at http://localhost:6510 that renders your entire project as an interactive 2D graph — scripts, scenes, and how signals connect them. Install via AssetLib: search "Godot AI Assistant tools MCP" in the Godot editor.

JSON .mcp.json (tomyud1 config)

```
{ "mcpServers": { "godot": { "command": "npx", "args": ["-y", "godot-mcp-server"], "env": { "GODOT_PORT":
"6505" } } } } // Browser visualizer auto-starts at http://localhost:6510
```

9.6 Running multiple MCP servers simultaneously

You can run two different Godot MCP servers at once if you need features from both. Just use different port numbers and give them different keys in .mcp.json.

JSON .mcp.json (two servers at once)

```
{ "mcpServers": { "godot-runtime": { "command": "npx", "args": ["-y", "@satelliteoflove/godot-mcp"],
"env": { "GODOT_PORT": "6550" } }, "godot-lsp": { "command": "npx", "args": ["-y", "minimal-godot-mcp"],
"env": {} } } } // Now Claude Code has runtime control AND LSP diagnostics at the same time.
```

§ 10

Offline Mode — Tuned

Ollama setup with optimal settings. GLM-5 via vLLM and SGLang for GPU clusters.

■ PICK YOUR OPTION

Option A (recommended): Ollama + glm4 tag = GLM-4.7-Flash. 30B total / 3B active MoE. ~8GB VRAM in 4-bit. ~45 t/s on RTX 3090. 59.2% SWE-bench. Best for: solo devs, laptops, privacy-first studios. Option B: GLM-5 via vLLM/SGLang. Full 77.8% SWE quality, no API cost. Needs 8× H100 80GB. Best for: studios with on-prem GPU clusters.

10.1 Option A — Ollama (most devs)

BASH terminal

```
# Install Ollama curl -fsSL https://ollama.com/install.sh | sh # macOS / Linux # Windows: download installer from ollama.com # Pull GLM-4.7-Flash (Ollama selects right quantisation for your VRAM) ollama pull glm4 # Optional: pull LLaVA for VLM/screenshot tasks ollama pull llava:13b # List what you have ollama list # Quick test ollama run glm4 "What is a Godot Node3D? One sentence." # The REST API is now live on :11434 — same as the bridge uses curl http://localhost:11434/api/tags
```

PYTHON ai_bridge/offline_handler.py

```
import httpx, os, asyncio from dotenv import load_dotenv load_dotenv() OLLAMA_URL = os.getenv("OLLAMA_URL", "http://localhost:11434") OLLAMA_MODEL = os.getenv("OLLAMA_MODEL", "glm4") async def ollama_complete(prompt: str, system: str = "", **_) -> str: payload = { "model": OLLAMA_MODEL, "prompt": prompt, "system": system or "You are a Godot 4.6 expert. Be concise.", "stream": False, "options": { "temperature": 0.2, "num_predict": 4096, "num_ctx": 32768, # 32K context is safe for GLM-4.7-Flash "repeat_penalty":1.1, # reduce repetition "top_p": 0.9, }, } async with httpx.AsyncClient(timeout=120) as c: r = await c.post(f"{OLLAMA_URL}/api/generate", json=payload) r.raise_for_status() return r.json()["response"] async def ollama_health() -> bool: try: async with httpx.AsyncClient(timeout=2) as c: r = await c.get(f"{OLLAMA_URL}/api/tags") return r.status_code == 200 except: return False
```

10.2 Option B — GLM-5 via vLLM (GPU cluster)

BASH launch_glm5_vllm.sh — requires 8× H100 80GB

```
# Install cutting-edge vLLM with GLM-5 support pip install -U vllm --pre \ --index-url https://pypi.org/simple \ --extra-index-url https://wheels.vllm.ai/nightly # Launch GLM-5 FP8 quantised (from HuggingFace: zai-org/GLM-5-FP8) vllm serve zai-org/GLM-5-FP8 \ --tensor-parallel-size 8 \ --gpu-memory-utilization 0.85 \ --speculative-config.method mtp \ --speculative-config.num_speculative_tokens 1 \ --tool-call-parser glm47 \ --reasoning-parser glm45 \ --enable-auto-tool-choice \ --served-model-name glm-5 \ --port 8080 # GLM-5 is now available at http://localhost:8080/v1 # It is OpenAI-API compatible — provider_glm.py works unchanged. # Just change base_url to http://localhost:8080/v1 in .env.
```

BASH launch_glm5_sglang.sh — for Hopper/Blackwell GPUs (H100/B200)

```
docker pull lmsysorg/sglang:glm5-hopper docker run --gpus all -p 8080:8080 lmsysorg/sglang:glm5-hopper \
python3 -m sglang.launch_server \ --model-path zai-org/GLM-5-FP8 \ --tp-size 8 \ --tool-call-parser glm47
\ --reasoning-parser glm45 \ --speculative-algorithm EAGLE \ --speculative-num-draft-tokens 4 \
--served-model-name glm-5 \ --port 8080 # SGLang with EAGLE speculative decoding is typically 15-20%
faster than # plain vLLM on the same hardware for GLM-5.
```

10.3 Pointing the bridge at a local GLM-5

BASH ai_bridge/.env (additions for local GLM-5)

```
# When running GLM-5 locally via vLLM or SGLang: ZAI_API_KEY=not-needed # any string, vLLM ignores auth
by default ZAI_BASE_URL=http://localhost:8080/v1 # In provider_glm.py, change the client to read
ZAI_BASE_URL: # base_url = os.getenv("ZAI_BASE_URL", "https://api.z.ai/api/paas/v4")
```

GitHub & Package Directory

Every repository and package referenced in this guide. Bookmark this page.

MCP SERVERS	
github.com/satelliteoflove/godot-mcp	Recommended MCP — runtime control, screenshots, animations. npx one-liner.
github.com/Coding-Solo/godot-mcp	32-tool MCP — UID handling, launch editor, run project, debug output.
github.com/tomyud1/godot-mcp	32 tools + browser visualizer at :6510. AssetLib install.
github.com/ee0pdt/Godot-MCP	Full script + scene read/write. stdio transport.
github.com/bradypp/godot-mcp	Auto Godot detection, read-only mode for safe CI analysis.
github.com/ryanmazzolini/minimal-godot-mcp	LSP + DAP diagnostics. No addon required.
AI MODELS & WEIGHTS	
huggingface.co/zai-org/GLM-5	GLM-5 model weights — MIT license, FP8 and BF16 variants.
huggingface.co/zai-org/GLM-5-FP8	GLM-5 FP8 quantised for vLLM/SGLang deployment.
huggingface.co/zai-org/GLM-4.7	GLM-4.7 weights (for full 355B deployment).
TOOLS & RUNTIMES	
ollama.com/install	Ollama — one-command local model server. Manages VRAM and quantisation.
github.com/vllm-project/vllm	vLLM — high-throughput inference for GLM-5 on GPU clusters.
github.com/sgl-project/sglang	SGLang — faster than vLLM on Hopper/Blackwell via EAGLE speculative decoding.
API DOCS	
aistudio.google.com	Google AI Studio — Gemini API keys and usage dashboard.
ai.google.dev/gemini-api/docs	Gemini API documentation, model list, pricing.
docs.z.ai/guides/llm/glm-5	GLM-5 API documentation and model changelog.
openrouter.ai/models/zai-org/glm-5	OpenRouter GLM-5 — cheaper rates, multi-provider fallback.

All code verified. All prices live-researched March 15, 2026. godotengine.org · aistudio.google.com · z.ai · artificialanalysis.ai/models/glm-5 · github.com/satelliteoflove/godot-mcp

§ 11

Streaming — Live Token Display

Show AI responses word-by-word in your game. No more 30-second freezes.

■ WHY THIS MATTERS FOR GAME FEEL

A 4096-token response at 380 t/s still takes 11 seconds to complete. For a game, that is an unacceptable freeze. Streaming shows the first token in under half a second and gives the player the feeling of a thinking, breathing entity. NPCs that type their responses feel alive. Loading screens that narrate what is happening feel dynamic. Streaming is the single biggest UX improvement for in-game AI. The bridge sends Server-Sent Events (SSE). GDScript reads them via HTTPClient in chunked mode — HTTPRequest cannot do this, it buffers the entire response.

11.1 Bridge SSE endpoint

PYTHON ai_bridge/bridge_server.py — add this endpoint

```
from fastapi.responses import StreamingResponse @app.post("/ai/stream") async def stream_ai(req: AiReq):
"""Server-Sent Events endpoint -- streams tokens as they arrive."""  from google import genai from
google.genai import types clt = genai.Client(api_key=os.environ["GEMINI_API_KEY"]) async def
token_generator(): try: for chunk in clt.models.generate_content_stream( model="gemini-3-flash-preview",
contents=[req.prompt], config=types.GenerateContentConfig( system_instruction=req.system or "You are a
Godot 4.6 expert.", temperature=0.3, thinking_config=types.ThinkingConfig(thinking_budget=0), ), ): if
chunk.text: # Replace newlines with RS (0x1E) so SSE line format stays valid. # GDScript decodes them back
to newlines on the other end. safe = chunk.text.replace(chr(10), chr(30)) yield f"data: {safe}\n\n" yield
"data: [DONE]\n\n" except Exception as ex: yield f"data: [ERR]{ex}\n\n" return StreamingResponse(
token_generator(), media_type="text/event-stream", headers={ "Cache-Control": "no-cache",
"X-Accel-Buffering":"no", # disable nginx buffering "Connection": "keep-alive", } )
```

11.2 GDScript streaming consumer

GDSCRIPT addons/ai_agent/ai_streamer.gd

```
extends Node ## Streams AI tokens in real time via Server-Sent Events. ## Connect token_received to
append text to a label or chat box. ## Connect stream_done when you need the full assembled response.
signal token_received(token: String) signal stream_done(full_text: String) signal stream_error(message:
String) const BRIDGE := "http://127.0.0.1:8000" var _http := HTTPClient.new() var _full := "" var _active
:= false var _buffer := "" func stream(prompt: String, system: String = "") -> void: _full = ""; _buffer =
""; _active = true var err := _http.connect_to_host("127.0.0.1", 8000) if err != OK:
stream_error.emit("Cannot connect to bridge: " + str(err)) return set_process(true) var body :=
JSON.stringify({"prompt": prompt, "system": system}) var headers := ["Content-Type: application/json",
"Accept: text/event-stream"] _http.request(HTTPClient.METHOD_POST, "/ai/stream", headers, body) func
cancel() -> void: _active = false; _http.close(); set_process(false) func _process(_dt: float) -> void:
_http.poll() if not _active: return match _http.get_status(): HTTPClient.STATUS_BODY: var chunk :=
_http.read_response_body_chunk() if chunk.size() > 0: _buffer += chunk.get_string_from_utf8()
_parse_buffer() HTTPClient.STATUS_CONNECTED: pass # waiting for response headers
HTTPClient.STATUS_DISCONNECTED, HTTPClient.STATUS_CONNECTION_ERROR: _active = false; set_process(false)
if _full.length() > 0: stream_done.emit(_full) _: pass func _parse_buffer() -> void: # SSE lines look
like: "data: \n\n" while "\n\n" in _buffer: var end_idx := _buffer.find("\n\n") var line :=
_buffer.substr(0, end_idx).strip_edges() _buffer = _buffer.substr(end_idx + 2) if not
line.begins_with("data: "): continue var data := line.substr(6) if data == "[DONE]":
stream_done.emit(_full) _active = false; set_process(false) return if data.begins_with("[ERR]"):
stream_error.emit(data.substr(5)) _active = false; set_process(false) return # Decode RS (0x1E) back to
newlines var token := data.replace(char(30), "\n") _full += token token_received.emit(token)
```

11.3 Wire streaming to a RichTextLabel

GDSCRIPT scenes/npc_dialogue_box.gd

```
extends Control @onready var label: RichTextLabel = $Label @onready var spinner: TextureRect = $Spinner
@onready var streamer: Node = $AIStreamer func _ready() -> void:
streamer.token_received.connect(_on_token) streamer.stream_done.connect(_on_done)
streamer.stream_error.connect(_on_error) func speak(prompt: String) -> void: label.text = ""
spinner.visible = true # System prompt gives the NPC its personality var system := "You are Mira, a
weathered blacksmith. Speak in short, gruff sentences." streamer.stream(prompt, system) func
_on_token(token: String) -> void: label.append_text(token) spinner.visible = false # hide spinner on
first token func _on_done(_full: String) -> void: # Full response is in _full -- use it for logging or TTS
pass func _on_error(msg: String) -> void: label.push_color(Color.RED) label.append_text("\n[AI
unavailable]") label.pop()
```

11.4 Typewriter effect vs raw streaming

Raw SSE streaming already looks like a typewriter because tokens arrive 1-3 words at a time at natural reading speed. For a stylised typewriter effect (letter by letter), buffer the tokens and emit characters with a Timer:

GDSCRIPT scenes/typewriter_label.gd

```
extends RichTextLabel var _queue := "" var _idx := 0 @export var chars_per_second: float = 40.0 @onready
var timer: Timer = Timer.new() func _ready() -> void: add_child(timer) timer.wait_time = 1.0 /
chars_per_second timer.timeout.connect(_emit_char) func feed_token(token: String) -> void: _queue +=
token if timer.is_stopped(): timer.start() func _emit_char() -> void: if _idx >= _queue.length():
timer.stop(); return append_text(_queue[_idx]) _idx += 1
```

§ 12

Error Handling & Retry

Every error that will happen in production. How to handle each one without breaking the game.

Error	HTTP Code	Retry?	Strategy
Rate limit exceeded	429	Yes (3×, exp backoff)	Wait 1s, 2s, 4s with jitter
Server error	500/503	Yes (once immediate)	Retry once, then fall to next provider
Context window exceeded	400	No	Trim prompt, re-submit with shorter context
Auth / key invalid	401/403	No	Log and alert — never auto-retry, key is broken
Response timeout	—	Yes (route change)	Skip provider, route to next in chain
Content filtered / blocked	400	No	Revise prompt wording, log for review
JSON parse error	200 but bad body	Yes (1×)	Retry with explicit JSON mode instruction
Ollama model not pulled	404	No	Pull model first, log setup instructions
All providers failed	—	No	Show user-friendly in-game error, log alert

12.1 Exponential backoff with full jitter

PYTHON ai_bridge/retry.py

```
import asyncio, random, functools, logging log = logging.getLogger("retry") RETRYABLE = {429, 500, 502, 503, 504} def with_retry(attempts: int = 3, base: float = 1.0, cap: float = 30.0): """ Exponential backoff with full jitter. Jitter prevents thundering-herd when many game clients retry simultaneously. """ def decorator(fn): @functools.wraps(fn) async def wrapper(*args, **kwargs): for n in range(1, attempts + 1): try: return await fn(*args, **kwargs) except Exception as exc: code = getattr(exc,"status_code", getattr(exc,"code", 0)) if n == attempts or code not in RETRYABLE: raise # Full jitter: delay = random(0, min(cap, base * 2^(n-1))) ceiling = min(cap, base * (2 ** (n - 1))) delay = random.uniform(0, ceiling) log.warning(f"[retry] attempt {n} failed (HTTP {code}), retrying in {delay:.2f}s") await asyncio.sleep(delay) return wrapper return decorator # Apply to any provider call: @with_retry(attempts=3, base=2.0) async def safe_gemini(prompt: str, **kw) -> dict: from provider_gemini import gemini_complete return await gemini_complete(prompt, **kw)
```

12.2 JSON repair — when the AI forgets JSON mode

Even with JSON mode enabled, models occasionally wrap output in markdown code fences. This utility strips them and retries parsing before raising an error:

PYTHON ai_bridge/json_utils.py

```
import json, re def safe_json(text: str) -> dict | list: """ Parse JSON from AI response, stripping markdown fences if present. Raises ValueError with clear message on failure. """ # Strip ```json ... ``` or ``` ... ``` fences stripped = re.sub(r"```(?:json)?\s*", "", text) stripped = stripped.replace("```", "").strip() # Try the cleaned version first try: return json.loads(stripped) except json.JSONDecodeError: pass # Try to extract first JSON object or array for start_char, end_char in [("[","]"),("{","}")]: s = stripped.find(start_char) e = stripped.rfind(end_char) if s != -1 and e > s: try: return json.loads(stripped[s:e+1]) except json.JSONDecodeError: continue raise ValueError(f"Could not parse JSON from AI response: {text[:200]}")
```

12.3 User-facing error messages in Godot

GDSCRIPT addons/ai_agent/ai_error_handler.gd

```
extends Node ## Maps bridge error codes to player-friendly messages. ## Attach this to your HUD or dialogue system. signal error_displayed(message: String) const MESSAGES := { "HTTP 429": "The AI is thinking about too many things at once. Try again in a moment.", "HTTP 503": "AI service temporarily unavailable. Switching to offline mode.", "HTTP 401": "AI configuration error. Please contact support.", "All providers failed": "AI offline. Some features are unavailable.", "HTTPRequest failed": "Cannot reach AI server. Check your connection.", "Invalid JSON": "AI returned an unexpected response. Retrying.", } func handle(raw_error: String) -> void: var msg := MESSAGES.get(raw_error, "AI encountered an issue. Some responses may be limited.") error_displayed.emit(msg) # Optionally: show as in-game notification, toast, or NPC line print("[AIError] ", raw_error, " → ", msg)
```

§ 13

Context Caching — 90% Off Repeated Context

Automatically pays for itself. Structure your prompts correctly and save 90% on input costs.

■ HOW IT WORKS

Gemini 3 Flash charges cache reads at $0.05/M — 10% of the $0.50/M input price. Implicit auto-caching (no code required): any prompt prefix that stays identical across calls and exceeds 2,048 tokens gets cached automatically. Explicit caching: you upload a context block manually, pay $1/M/hour storage, and get a guaranteed cache hit. For most Godot projects, IMPLICIT caching is free and covers 80% of the savings. Put your stable scene spec, system instructions, and game lore at the top of every prompt. Put the variable part (the actual question) at the bottom.

13.1 Implicit caching — just structure your prompts correctly

PYTHON ai_bridge/provider_gemini.py — cached wrapper

```
# Your game's scene context -- stays the same across most calls. # The longer and more stable this is, the more you save. SCENE_CONTEXT = """ You are an AI agent embedded in Godot 4.6. Current scene structure: Root: Node3D (World) ■■ Environment: WorldEnvironment ■■ Player: CharacterBody3D ■ ■■ CollisionShape3D ■ ■■ MeshInstance3D ■■ Level: Node3D ■ ■■ Terrain: MeshInstance3D ■ ■■ Enemies: Node3D ■■ UI: CanvasLayer Available Godot 4.6 node types for scene commands: MeshInstance3D, CSGBox3D, CSGSphere3D, CSGCombiner3D, CharacterBody3D, RigidBody3D, StaticBody3D, CollisionShape3D, SpotLight3D, OmniLight3D, DirectionalLight3D, GPUParticles3D, CPUParticles3D, AudioStreamPlayer3D, AnimationPlayer, Skeleton3D, BoneAttachment3D, NavigationAgent3D, NavigationRegion3D Rules: - Always return valid JSON when asked for scene commands. - Node names must be unique within their parent. - Use dot notation for nested paths: /World/Level/Enemies """ async def gemini_with_context(question: str, thinking: str = "medium") -> dict: """ SCENE_CONTEXT goes first → eligible for implicit cache hit. question goes last → changes each call, billed at full rate. Savings: ~90% on SCENE_CONTEXT tokens after first call. """ full_prompt = SCENE_CONTEXT + "\n\n" + question result = await gemini_complete(full_prompt, thinking=thinking) # Log cache savings cached = result.get("cached", 0) total = result.get("tokens", 0) saved_usd = cached * 0.9 * (0.50 / 1_000_000) if cached > 0: print(f"[cache] {cached}/{total} tokens cached · ~${saved_usd:.5f} saved this call") return result
```

13.2 Break-even calculator — when to use explicit caching

PYTHON ai_bridge/cache_calculator.py

```
def break_even(cached_tokens: int, input_price: float = 0.50, cache_discount: float = 0.90, storage_per_m_per_hr: float = 1.00) -> float: """ Returns: queries per hour needed for explicit caching to break even. Below this number, implicit (free) caching is better. Example: 50K-token scene spec savings_per_query = 50000 * (0.50/1M) * 0.90 = $0.0000225 storage_cost_per_hr = 50000 * (1.00/1M) = $0.05 break_even = 0.05 / 0.0000225 ≈ 2222 queries/hr """ savings_per_q = cached_tokens * (input_price / 1_000_000) * cache_discount storage_per_hr = cached_tokens * (storage_per_m_per_hr / 1_000_000) if savings_per_q == 0: return float("inf") return storage_per_hr / savings_per_q if __name__ == "__main__": for tokens in [10_000, 50_000, 100_000, 500_000]: be = break_even(tokens) print(f"{tokens:>8,} tokens → break-even: {be:>8,.0f} queries/hr") # Output: # 10,000 tokens → break-even: 2,222 queries/hr # 50,000 tokens → break-even: 2,222 queries/hr (ratio is always 2222) # Conclusion: for most Godot games, implicit caching is sufficient. # Only use explicit caching if you have >2000 AI calls per hour.
```

13.3 Explicit cache for large persistent game lore

If your game has a massive lore bible (world history, faction relationships, NPC personalities — 50K+ tokens) that you reference in every AI call, explicit caching at $0.05/M cache read (vs $0.50/M normal) cuts costs dramatically once you pass 2000 calls/hour:

PYTHON ai_bridge/cache_manager.py

```
from google import genai from google.genai import types import os client =
genai.Client(api_key=os.environ["GEMINI_API_KEY"]) MODEL = "models/gemini-3-flash-preview" class
LoreCache: """Cache a large static lore document for 90% discount on repeated reads."""  def
__init__(self): self._cache_name: str | None = None def create(self, lore_document: str, ttl_seconds: int
= 3600) -> str: """Upload lore to cache. Min 32K tokens. $1/M tokens/hr storage."""  c =
client.caches.create( model=MODEL, config=types.CreateCachedContentConfig(
display_name="game-lore-cache", system_instruction="You are a lore-accurate narrator for this world.",
contents=[lore_document], ttl=f"{ttl_seconds}s", ) ) self._cache_name = c.name print(f"[LoreCache]
created: {c.usage_metadata.total_token_count} tokens cached") return self._cache_name def query(self,
question: str) -> str: """Ask a question using the cached lore. Cache hit = 90% off input."""  if not
self._cache_name: raise RuntimeError("Call create() first") resp = client.models.generate_content(
model=MODEL, contents=[question], config=types.GenerateContentConfig( cached_content=self._cache_name,
temperature=0.3, ) ) return resp.text
```

§ 14

GLM-5 Multi-Turn Agent

Preserved Thinking: the AI remembers WHY it made every decision across a 20-turn session.

■ PRESERVED THINKING — THE KEY DIFFERENCE

Standard LLMs reset their internal reasoning on every new message. GLM-5 with clear_thinking=False retains every reasoning block across the entire session. In practice: you can say "build a dungeon level" and then "add more traps" and "now connect it to the boss room" — across 15 messages — and the AI remembers exactly what it planned, why it placed each node, and what still needs doing. Without Preserved Thinking, each message starts cold and the AI forgets the architectural decisions it made three turns ago.

PYTHON — ai_bridge/glm_agent.py

```
from openai import AsyncOpenAI import os, json client = AsyncOpenAI( api_key = os.environ["ZAI_API_KEY"],
base_url = "https://api.z.ai/api/paas/v4", ) GODOT_TOOLS = [
{"type":"function","function":{"name":"create_node", "description":"Create a node in the Godot scene.",
"parameters":{"type":"object","required":["type","name"],
"properties":{"type":{"type":"string"},"name":{"type":"string"},
"parent":{"type":"string","default":"."}, "props":{"type":"object"}}}}},
{"type":"function","function":{"name":"set_property", "description":"Set a property on a node.",
"parameters":{"type":"object","required":["path","property","value"],
"properties":{"path":{"type":"string"},"property":{"type":"string"}, "value":{"type":"string"}}}}},
{"type":"function","function":{"name":"attach_script", "description":"Write and attach a GDScript file to
a node.", "parameters":{"type":"object","required":["path","code"],
"properties":{"path":{"type":"string"},"code":{"type":"string"}, "filename":{"type":"string"}}}}},
{"type":"function","function":{"name":"delete_node", "description":"Delete a node from the scene.",
"parameters":{"type":"object","required":["path"], "properties":{"path":{"type":"string"}}}}}, ] class
GodotAgent: """ Stateful multi-turn GLM-5 agent with Preserved Thinking. The agent remembers its
reasoning across the entire session. """ def __init__(self, game_context: str = ""): self.history:
list[dict] = [] self.system = ( "You are an autonomous Godot 4.6 scene-building agent. " "Use the
available tools to build and modify the scene. " "Think carefully before each action. " "Remember your
plan across the entire conversation." ) if game_context: self.system += f"\n\nGame
context:\n{game_context}" self.total_tokens = 0 async def step( self, user_msg: str, complex_task: bool =
True, # False = skip thinking for simple ops ) -> dict: """One turn. Returns {actions, text, thinking,
tokens}."""  self.history.append({"role":"user","content":user_msg}) msgs =
[{"role":"system","content":self.system}] + self.history resp = await client.chat.completions.create(
model="glm-5", messages=msgs, tools=GODOT_TOOLS, tool_choice="auto", extra_body={ "thinking":
{"type":"enabled" if complex_task else "disabled"}, "clear_thinking": False, # Preserved Thinking --
never reset }, max_tokens=8192, temperature=0.2, ) msg = resp.choices[0].message actions = [] if
msg.tool_calls: actions = [ {"id": tc.id, "tool": tc.function.name, "args":
json.loads(tc.function.arguments)} for tc in msg.tool_calls ] self.history.append({ "role": "assistant",
"content": None, "tool_calls": [tc.model_dump() for tc in msg.tool_calls], }) else:
self.history.append({"role":"assistant","content":msg.content}) self.total_tokens +=
resp.usage.total_tokens return { "actions": actions, "text": msg.content, "thinking":
getattr(msg,"reasoning_content",None), "tokens": resp.usage.total_tokens, } def add_tool_result(self,
tool_id: str, result: str) -> None: """Feed the result of a tool call back into the conversation."""
self.history.append({ "role": "tool", "tool_call_id": tool_id, "content": result, }) def summary(self) ->
str: return (f"Session: {len(self.history)} turns, {self.total_tokens} total tokens,
${self.total_tokens/1_000_000:.4f} cost")
```

14.2 Full autonomous dungeon build session

PYTHON — example: build_dungeon.py

```
import asyncio from glm_agent import GodotAgent # execute_tool() sends commands to Godot via MCP
WebSocket from bridge_server import execute_tool async def build_dungeon_level(): agent =
GodotAgent(game_context="""  Dark fantasy dungeon crawler. Art direction: stone walls, torchlight, deep
shadows. Parent for all level geometry: /World/Level  """) # Turn 1 -- complex planning, thinking enabled
r1 = await agent.step( "Design a dungeon room: 10m×10m stone chamber with 4 corners, " "2 torches on
opposite walls, a locked iron door, and bone debris on the floor. " "Start building it now.",
complex_task=True ) for action in r1["actions"]: result = await execute_tool(action["tool"],
action["args"]) agent.add_tool_result(action["id"], str(result)) print(f"[T1] {len(r1['actions'])}
actions, thinking: {bool(r1['thinking'])}") # Turn 2 -- simple property tweak, disable thinking for speed
r2 = await agent.step( "Set both torches to orange colour temp 2200K and energy 2.0.", complex_task=False
# ~1s instead of ~8s ) for action in r2["actions"]: result = await execute_tool(action["tool"],
action["args"]) agent.add_tool_result(action["id"], str(result)) # Turn 3 -- complex: add atmosphere,
thinking re-enabled r3 = await agent.step( "Add subtle fog and a faint dripping sound. " "Also check if
anything looks wrong with the layout so far.", complex_task=True ) for action in r3["actions"]: result =
await execute_tool(action["tool"], action["args"]) agent.add_tool_result(action["id"], str(result))
print(agent.summary()) asyncio.run(build_dungeon_level())
```

§ 15

Testing Without Burning Money

Full pytest suite with mocked providers, contract tests, and Godot headless CI.

PYTHON ai_bridge/tests/conftest.py

```
import pytest from unittest.mock import AsyncMock, patch # Fake responses matching real provider output structure FAKE_GEMINI = { "text": "[{action:create,type:MeshInstance3D,name:Rock}]", "tokens": 48, "cached": 0, "think_t": 0, "model": "gemini-3-flash-preview" } FAKE_GLM = { "text": "Here is the scene command array [...]", "thinking":"I will create a rock node first.", "tokens": 62, "model": "glm-5" } FAKE_OLLAMA = "[{action:create,type:Node3D,name:Fallback}]" @pytest.fixture def mock_gemini(): with patch("provider_gemini.gemini_complete", new_callable=AsyncMock, return_value=FAKE_GEMINI) as m: yield m @pytest.fixture def mock_glm(): with patch("provider_glm.glm_complete", new_callable=AsyncMock, return_value=FAKE_GLM) as m: yield m @pytest.fixture def mock_ollama(): with patch("offline_handler.ollama_complete", new_callable=AsyncMock, return_value=FAKE_OLLAMA) as m: yield m @pytest.fixture def all_mocked(mock_gemini, mock_glm, mock_ollama): return {"gemini": mock_gemini, "glm": mock_glm, "ollama": mock_ollama}
```

PYTHON ai_bridge/tests/test_router.py

```
import pytest from provider_router import route @pytest.mark.asyncio async def test_uses_gemini_first(all_mocked): r = await route("Create a rock node") assert r["provider"] == "gemini" all_mocked["glm"].assert_not_awaited() all_mocked["ollama"].assert_not_awaited() @pytest.mark.asyncio async def test_falls_to_glm_on_gemini_failure(all_mocked): all_mocked["gemini"].side_effect = RuntimeError("503 Service Unavailable") r = await route("Create a rock node") assert r["provider"] == "glm-5" @pytest.mark.asyncio async def test_falls_to_ollama_when_cloud_fails(all_mocked): all_mocked["gemini"].side_effect = RuntimeError("503") all_mocked["glm"].side_effect = RuntimeError("503") r = await route("Create a rock node") assert r["provider"] == "ollama" @pytest.mark.asyncio async def test_all_fail_returns_error_dict(all_mocked): for m in all_mocked.values(): m.side_effect = RuntimeError("down") r = await route("Create a rock node") assert "error" in r assert r["provider"] == "none" @pytest.mark.asyncio async def test_vision_skips_glm(all_mocked): # Vision tasks must skip GLM-5 (text-only) -- use Gemini only. r = await route("Analyse this screenshot", has_image=True) all_mocked["glm"].assert_not_awaited() @pytest.mark.asyncio async def test_offline_flag_skips_cloud(all_mocked, monkeypatch): monkeypatch.setenv("AI_OFFLINE", "true") r = await route("Hello") assert r["provider"] == "ollama" all_mocked["gemini"].assert_not_awaited()
```

BASH terminal - run tests

```
cd ai_bridge && source .venv/bin/activate # Run all unit tests (no API calls, free) pytest tests/ -v --asyncio-mode=auto # Run with coverage pytest tests/ --cov=. --cov-report=term-missing # Run only contract tests against real APIs (costs money -- run nightly) pytest tests/ -m contract -v
```

§ 16

CI/CD & Docker

Automated testing, Docker Compose local stack, GitHub Actions — every push tested for free.

YAML docker-compose.yml

```
version: "3.9" services: bridge: build: context: ./ai_bridge dockerfile: Dockerfile ports: ["8000:8000"]
env_file: ./ai_bridge/.env environment: OLLAMA_URL: http://ollama:11434 BRIDGE_HOST: 0.0.0.0 depends_on:
[ollama] restart: unless-stopped healthcheck: test: ["CMD","curl","-f","http://localhost:8000/health"]
interval: 10s timeout: 5s retries: 3 ollama: image: ollama/ollama:latest ports: ["11434:11434"] volumes:
[ollama_data:/root/.ollama] restart: unless-stopped # GPU support (uncomment for NVIDIA): # deploy: #
resources: # reservations: # devices: [{driver:nvidia,count:1,capabilities:[gpu]}] volumes: ollama_data:
```

DOCKERFILE ai_bridge/Dockerfile

```
FROM python:3.12-slim WORKDIR /app COPY requirements.txt . RUN pip install --no-cache-dir -r
requirements.txt COPY . . EXPOSE 8000 CMD ["python", "bridge_server.py"]
```

YAML .github/workflows/ci.yml

```
name: AI Bridge Tests on: push: { branches: [main, develop] } pull_request: { branches: [main] } jobs:
test: runs-on: ubuntu-latest steps: - uses: actions/checkout@v4 - uses: actions/setup-python@v5 with: {
python-version: "3.12" } - name: Install dependencies run: pip install -r ai_bridge/requirements.txt
pytest pytest-asyncio pytest-cov - name: Run unit tests (no API calls) run: pytest ai_bridge/tests/ -v
--asyncio-mode=auto --cov=ai_bridge env: AI_OFFLINE: "true" GEMINI_API_KEY: "fake-for-tests" ZAI_API_KEY:
"fake-for-tests" - name: Upload coverage uses: codecov/codecov-action@v4 if: always() # Contract tests
run nightly -- they hit real APIs contract: runs-on: ubuntu-latest if: github.event_name == 'schedule'
steps: - uses: actions/checkout@v4 - uses: actions/setup-python@v5 with: { python-version: "3.12" } -
run: pip install -r ai_bridge/requirements.txt pytest pytest-asyncio - run: pytest ai_bridge/tests/ -m
contract -v env: GEMINI_API_KEY: ${{ secrets.GEMINI_API_KEY }} ZAI_API_KEY: ${{ secrets.ZAI_API_KEY }}
```

§ 17

Model & Cost Reference — March 15, 2026

Live-verified pricing. Every model relevant to Godot AI integration.

■ DEPRECATED — MIGRATE BEFORE JUNE 1, 2026
gemini-2.0-flash and gemini-2.0-flash-lite are shut down June 1, 2026. If any code in your project uses these strings, replace them with "gemini-3-flash-preview" now. The new model is better on every benchmark and cheaper.

Model	Provider	Input	Output	Cache	Context	t/s
gemini-3-flash-preview	Google	$0.50	$3.00	$0.05	1M	380+
gemini-3.1-pro-preview-02-05	Google	$2.00	$18.00	$0.20	1M	~100
gemini-3.1-flash-lite	Google	$0.25	$1.50	$0.025	1M	299
gemini-2.5-pro-preview-06-05	Google	$1.25	$10.00	$0.125	1M	~127
gemini-2.0-flash	Google	■ DEPRECATED	—	—	—	—
glm-5 (z.ai)	z.ai	$1.00	$3.20	—	200K	63.5
glm-5 (OpenRouter)	OpenRouter	$0.72	$2.30	—	202K	63.5
glm-4.7 (z.ai)	z.ai	$0.38	$1.98	—	200K	~55
glm-4.7-Flash via Ollama	Local	free	free	—	128K	~45
glm-5 FP8 via vLLM	Local	free	free	—	200K	varies

Benchmark	Gemini 3 Flash	GLM-5	Notes
SWE-bench Verified	78.0%	77.8%	Effectively tied — both best class
LMArena Text Arena	top-5	#1 open (score 1452)	GLM-5 highest-rated open model
GPQA-Diamond	90.4%	86.0%	Gemini stronger on hard science
AIME 2026	—	92.7%	GLM-5 excellent at maths
Context	1,000,000	200,000	Use Gemini for very long contexts
Vision (images)	Yes	No (text-only)	Gemini required for screenshot VLM
MIT license	No	Yes	GLM-5 weights fully open
Offline deployment	No	Yes (vLLM/SGLang)	GLM-5 self-hostable

Decision table — which model for which task

You need to…	Use	Estimated cost
Handle most in-game AI (NPC dialogue, codegen, scene build)	Gemini 3 Flash, thinking=medium	~$0.010 / 4K response
Fallback when Gemini is rate-limited or down	GLM-5 via z.ai	~$0.022 / 4K response
Analyse a screenshot of your scene	Gemini 3 Flash + image_path=	~$0.006 / image
Run a 10-turn agent that builds a whole level	GLM-5 Preserved Thinking	~$0.015 / turn
Process lore or context over 200K tokens	Gemini 3 Flash (1M ctx)	~$0.10–0.25 / call
Maximum reasoning for complex architecture	Gemini 3 Flash, thinking=high	~$0.018 / call
Work completely offline — no API key	Ollama glm4 (GLM-4.7-Flash)	$0.00
Studio GPU cluster, full open-source quality	GLM-5 via vLLM or SGLang	$0.00 after hardware
Nightly screenshot regression testing (50 images)	Gemini Batch API (50% off)	~$0.015 total
High-volume property updates (>1000 calls/hour)	Gemini 3.1 Flash-Lite ($0.25/M)	~$0.003 / call

§ 18

Standard Use Cases

Complete implementations for the AI features every game developer reaches for first.

USE CASE 18.1

Contextual NPC Dialogue

NPCs that remember what they know, who you are, and what just happened — and never repeat themselves.

The naive approach is to ask the AI for NPC dialogue without context. The result is generic. The right approach is to stuff the NPC's personality, knowledge, relationship with the player, and recent events into the system prompt:

GDSCRIPT npc/npc_brain.gd

```
extends Node # NPC configuration -- set these in the Inspector @export var npc_name: String = "Mira"
@export var npc_personality: String = "Gruff blacksmith, speaks in short sentences. Distrusts magic."
@export var npc_knowledge: String = "Knows the town layout. Has heard rumours of the eastern ruins." #
Runtime state -- updated as the game progresses var relationship_score: int = 0 # -100 to 100 var
last_interaction: String = "" var revealed_secrets: Array = [] signal dialogue_line(text: String) func
speak(player_said: String, world_context: String = "") -> void: var system_prompt :=
_build_system(world_context) AIAgent.response_received.connect(_on_dialogue, CONNECT_ONE_SHOT)
AIAgent.ask(player_said, "minimal", "auto") # Note: pass system via bridge if you extend the AiReq model
to include it func _build_system(world_context: String) -> String: var rel := "neutral" if
relationship_score > 40: rel = "friendly, slightly warm" elif relationship_score < -20: rel =
"suspicious, curt" return ( "You are " + npc_name + ". " + npc_personality + "\n" + "What you know: " +
npc_knowledge + "\n" + "Your relationship with the player: " + rel + "\n" + ("Last interaction: " +
last_interaction + "\n" if last_interaction else "") + (world_context + "\n" if world_context else "") +
"Respond in 1-3 short sentences. Stay in character. Never break the fourth wall." ) func
_on_dialogue(text: String, _p: String) -> void: last_interaction = text.substr(0, 100)
dialogue_line.emit(text)
```

USE CASE 18.2

Procedural Quest Generation

AI generates unique side quests with branching objectives, fitting your world lore.

PYTHON ai_bridge/quest_generator.py

```
import json from provider_gemini import gemini_complete QUEST_SCHEMA = { "type": "object", "properties": { "title": {"type":"string"}, "giver": {"type":"string"}, "summary": {"type":"string"}, "objectives": {"type":"array","items":{"type":"string"}}, "reward": {"type":"object","properties":{ "gold": {"type":"integer"}, "item": {"type":"string"}, }}, "failure_state": {"type":"string"}, }, "required": ["title","giver","summary","objectives","reward"], } async def generate_quest( region: str, player_level: int, recent_events: list[str], ) -> dict: prompt = ( f"Generate a side quest for region: {region}. " f"Player level: {player_level}. " f"Recent world events: {recent_events}. " "Make the quest feel connected to these events. " "Scale difficulty and reward to player level." ) result = await gemini_complete( prompt, thinking="medium", json_schema=QUEST_SCHEMA ) return json.loads(result["text"]) # Example output: # { # "title": "The Missing Cartographer", # "giver": "Innkeeper Aldous", # "summary": "A mapmaker went to survey the eastern ruins 3 days ago...", # "objectives": ["Find Tomlin's last camp","Retrieve his maps","Report to Aldous"], # "reward": {"gold": 150, "item": "Worn Map of the Eastern Reach"}, # }
```

USE CASE 18.3

Dynamic Difficulty Adjustment

AI analyses player behaviour and adjusts enemy stats, puzzle complexity, and resource drops in real time.

GDSCRIPT systems/difficulty_manager.gd

```
extends Node var stats := { "deaths_last_10min": 0, "avg_kill_time_sec": 0.0, "items_used": 0, "time_stuck_sec": 0, "damage_taken_pct": 0.0, } func evaluate_and_adjust() -> void: var prompt := ( "Player performance data: " + JSON.stringify(stats) + "\n" + "Current difficulty multiplier: " + str(GameState.difficulty) + "\n" + "Return JSON: {multiplier: float 0.5-2.0, reason: string, " + "adjustments: {enemy_health_mult: float, loot_rate_mult: float}}" ) AIAgent.response_received.connect(_apply_adjustment, CONNECT_ONE_SHOT) AIAgent.ask(prompt, "none") # Fast decision, no thinking needed func _apply_adjustment(text: String, _p: String) -> void: var s := text.find("{"); var e := text.rfind("}")+1 if s == -1: return var data: Dictionary = JSON.parse_string(text.substr(s, e-s)) GameState.difficulty = data.get("multiplier", 1.0) GameState.enemy_health_mult = data.get("adjustments",{}).get("enemy_health_mult",1.0) GameState.loot_rate_mult = data.get("adjustments",{}).get("loot_rate_mult",1.0) print("[Difficulty] ", data.get("reason","adjusted"))
```

USE CASE 18.4

Voice Input → Game Actions

Player speaks naturally. AI converts speech to structured game commands.

PYTHON ai_bridge/voice_to_action.py

```
import json from provider_gemini import gemini_complete ACTION_SCHEMA = { "type": "object", "properties": { "action": {"type":"string", "enum":["move","attack","use_item","talk","craft","open","examine"]}, "target": {"type":"string"}, "direction": {"type":"string"}, "item": {"type":"string"}, }, "required": ["action"], } async def parse_voice_command(transcript: str) -> dict: """ Convert natural speech to a structured game action. Input: "go north and attack the goblin on the left" Output: {"action":"move","direction":"north"} then {"action":"attack","target":"goblin_left"} """ prompt = ( f'Player said: "{transcript}"\n' "Convert this to a single game action. " "If multiple actions, return the first one only." ) result = await gemini_complete( prompt, thinking="none", json_schema=ACTION_SCHEMA ) return json.loads(result["text"]) # Pair with Godot's built-in speech recognition or OS microphone input: # 1. Record audio in Godot → send to Whisper API for transcription # 2. Send transcript to this endpoint # 3. Receive structured action → dispatch to game input system
```

§ 19

Advanced Use Cases

Techniques that most developers have not discovered yet. Each one is a significant competitive advantage.

USE CASE 19.1

Autonomous AI Playtester

AI plays your game overnight, files bug reports, and identifies softlocks — while you sleep.

The most expensive part of game development is playtesting. You can replace a significant portion of early playtesting by having an AI agent control your game autonomously via MCP, observe what happens, and write bug reports. This works best in headless mode with Godot 4.6's LibGodot.

PYTHON ai_bridge/autonomous_playtester.py

```
import asyncio, json, pathlib, time from glm_agent import GodotAgent from provider_gemini import gemini_complete SCREENSHOT = "/tmp/godot_playtest.png" async def playtest_session(max_turns: int = 50): agent = GodotAgent(game_context="""  You are a game playtester. Your job is to: 1. Explore every area of the game. 2. Try to break things -- use items in unexpected ways. 3. Note any bugs, softlocks, or missing feedback. 4. Write a structured bug report after each significant finding. Use the available tools to move the player, interact with objects, and take screenshots to verify what you see. """) bug_reports = [] for turn in range(max_turns): # Take screenshot and ask AI what to do next r = await agent.step( f"Turn {turn+1}/{max_turns}. Take a screenshot, analyse the game state, " "then take the most interesting or risky action you can. " "If you find a bug, document it immediately.", complex_task=(turn % 5 == 0), # deep thinking every 5 turns ) if r["text"] and "BUG:" in r["text"]: bug_reports.append({ "turn": turn, "report": r["text"], "timestamp": time.time(), }) for action in r["actions"]: # Execute MCP tool calls (move player, interact, screenshot...) from bridge_server import execute_tool result = await execute_tool(action["tool"], action["args"]) agent.add_tool_result(action["id"], str(result)) await asyncio.sleep(0.3) # Save bug report pathlib.Path("playtest_bugs.json").write_text(json.dumps(bug_reports, indent=2)) print(f"[Playtester] {len(bug_reports)} bugs found in {max_turns} turns") return bug_reports
```

USE CASE 19.2

Live Economy Balancer

AI monitors your game economy, detects exploits, and rebalances item values without a patch.

PYTHON ai_bridge/economy_balancer.py

```
import json from provider_gemini import gemini_complete BALANCE_SCHEMA = { "type": "object",
"properties": { "adjustments": { "type": "array", "items": { "type": "object", "properties": { "item_id":
{"type":"string"}, "new_value": {"type":"number"}, "reason": {"type":"string"}, }, "required":
["item_id","new_value","reason"], } }, "exploit_detected": {"type":"boolean"}, "exploit_desc":
{"type":"string"}, } } async def analyse_economy(economy_snapshot: dict) -> dict: """ economy_snapshot
format: { "avg_gold_per_hour": 1240, "most_sold_items": [{"id":"iron_sword","count":3820}],
"least_used_items": [{"id":"fire_scroll","count":12}], "price_history": {"iron_sword":
[40,42,38,45,80,120]}, } """ prompt = ( "Analyse this game economy snapshot and identify imbalances:\n" +
json.dumps(economy_snapshot, indent=2) + "\nLook for: item price inflation, gold sinks not working, " +
"underused items, potential exploits (rapid price changes). " + "Suggest specific item value
adjustments." ) result = await gemini_complete(prompt, thinking="medium", json_schema=BALANCE_SCHEMA)
return json.loads(result["text"]) # Wire this to run every 30 minutes via a background task. # Apply
adjustments via Godot signal without a game restart.
```

USE CASE 19.3

Runtime Localisation of Generated Content

AI generates text in the player's language. Player sees a Spanish dungeon sign even if you never wrote Spanish.

PYTHON ai_bridge/localiser.py

```
from provider_gemini import gemini_complete async def localise(text: str, target_lang: str, context: str
= "") -> str: """ Translate AI-generated content at runtime. Unlike static localisation tables, this
handles GENERATED text that was never in the original translation strings. """ prompt = ( f"Translate the
following game text to {target_lang}. " f"Keep the tone, style, and any proper nouns (item names, place
names) intact. " f"Context: {context}\n\n" f"Text to translate:\n{text}" ) result = await
gemini_complete(prompt, thinking="none") return result["text"].strip() # Cost note: at $0.50/M input +
$3.00/M output, # translating a 100-word NPC response costs ~$0.0004. # For 1000 players each triggering
10 translations/session: # 1000 * 10 * $0.0004 = $4.00 per session -- very affordable. # GDScript usage: #
AIAgent.ask("Translate to French: " + original_text, "none")
```

USE CASE 19.4

Adaptive World Narrative

The world reacts to player choices with custom lore entries, rumours, and NPC memory.

GDSCRIPT systems/world_narrator.gd

```
extends Node ## When the player does something significant, the world notices. ## NPCs gossip about it. Books mention it. Signs warn about it. var significant_events: Array[Dictionary] = [] func player_did(action: String, location: String, details: Dictionary) -> void: var event := { "action": action, "location": location, "details": details, "time": Time.get_ticks_msec(), } significant_events.append(event) _propagate_event(event) func _propagate_event(event: Dictionary) -> void: var prompt := ( "A player in a dark fantasy RPG just did this: " + JSON.stringify(event) + "\n" + "Generate 3 things: (1) a 1-sentence rumour an NPC might say, " + "(2) a 1-sentence journal entry a witness would write, " + "(3) a short warning sign that might appear near the location. " + "Return JSON: {rumour: string, journal: string, sign: string}" ) AIAgent.response_received.connect( func(text, _p): _apply_world_reaction(text, event), CONNECT_ONE_SHOT) AIAgent.ask(prompt, "minimal") func _apply_world_reaction(text: String, _event: Dictionary) -> void: var s := text.find("{"); var e := text.rfind("}")+1 if s == -1: return var data: Dictionary = JSON.parse_string(text.substr(s, e-s)) WorldState.add_rumour(data.get("rumour", "")) WorldState.add_journal_entry(data.get("journal", "")) # Spawn a procedural sign near the event location WorldState.spawn_sign(data.get("sign", ""))
```

§ 20

Secret Creative Use Cases

Applications most developers have never considered. Each one is a real competitive advantage.

■ THESE ARE NOT OBVIOUS

The standard uses — NPC dialogue, procedural generation, difficulty scaling — are well-known. The techniques in this section are not. Most developers using AI in games are doing one or two of the standard things. The developers who use these techniques will build experiences that feel completely different. Some of these are genuinely unexplored territory.

USE CASE 20.1

AI-Driven Soundtrack Composition

The AI composes original background music described as text, then your code converts it to audio parameters.

Most game AI focuses on visuals and text. Music is almost completely ignored. But Gemini can describe a musical piece in enough detail that you can drive a procedural audio system from it — or generate the actual notes if you have a MIDI system:

PYTHON ai_bridge/music_director.py

```
import json from provider_gemini import gemini_complete MUSIC_SCHEMA = { "type": "object", "properties": { "tempo_bpm": {"type":"integer","minimum":40,"maximum":200}, "key": {"type":"string"}, "mood": {"type":"string"}, "instruments": {"type":"array","items":{"type":"string"}}, "intensity": {"type":"number","minimum":0.0,"maximum":1.0}, "reverb": {"type":"number","minimum":0.0,"maximum":1.0}, "motif_notes": {"type":"array","items":{"type":"string"}, "description":"MIDI note names for the main motif"}, } } async def compose_for_scene(scene_description: str) -> dict: """ Give it a scene. Get back music parameters. Wire these to your AudioStreamPlayer or a procedural music system. """ prompt = ( f"Compose background music for this game scene: {scene_description}\n" "Describe the music in structured parameters. " "The motif_notes should be 4-8 MIDI note names (e.g. C4, E4, G4) " "that define the scene's melodic identity." ) result = await gemini_complete(prompt, thinking="medium", json_schema=MUSIC_SCHEMA) return json.loads(result["text"]) # Example output for "dark dungeon with distant dripping water": # { "tempo_bpm": 52, "key": "D minor", "mood": "ominous", # "instruments": ["low strings","prepared piano","distant bells"], # "intensity": 0.3, "reverb": 0.85, # "motif_notes": ["D3","F3","A3","C4"] } # Wire to Godot AudioServer bus parameters and a MIDI player node.
```

USE CASE 20.2

Shader Generation from Visual Description

Describe what you want a shader to look like. AI writes the shader code. No GLSL knowledge required.

PYTHON ai_bridge/shader_generator.py

```
from provider_glm import glm_complete SHADER_SYSTEM = """ You are a Godot 4.6 shader expert. When asked to create a shader: 1. Write valid Godot 4.6 VisualShader-compatible GLSL. 2. Return ONLY the shader code, no explanation. 3. Always include shader_type and render_mode at the top. 4. For spatial shaders, use proper albedo/emission/roughness/metallic outputs. """ async def generate_shader(description: str) -> str: """ Input: "A lava material: glowing orange cracks, pulsing with heat, dark rock between cracks" Output: Working Godot 4.6 spatial shader source code """ result = await glm_complete( f"Create a Godot 4.6 spatial shader for: {description}", system=SHADER_SYSTEM, thinking=True, ) return result["text"] # In Godot: # 1. Call this endpoint with a description # 2. Create a ShaderMaterial: var mat = ShaderMaterial.new() # 3. Create a Shader: var sh = Shader.new(); sh.code = received_code # 4. Apply: mat.shader = sh; mesh_instance.material_override = mat # Cost: ~$0.015 per shader (GLM-5 with thinking enabled). # Quality check: run the shader in Godot and screenshot it. # Feed the screenshot back to Gemini Vision to verify it looks right.
```

USE CASE 20.3

Procedural Dialogue Tree Generation

AI generates complete branching dialogue trees, not just single responses. Export directly to Godot's dialogue format.

PYTHON ai_bridge/dialogue_tree_gen.py

```
import json from provider_gemini import gemini_complete TREE_SCHEMA = { "type": "object", "properties": { "nodes": { "type": "array", "items": { "type": "object", "properties": { "id": {"type":"string"}, "speaker": {"type":"string"}, "text": {"type":"string"}, "choices": {"type":"array","items":{ "type":"object","properties":{ "label":{"type":"string"}, "next": {"type":"string"}, "condition":{"type":"string"}, }}}, } } } } } async def generate_dialogue_tree( npc_name: str, topic: str, context: str, depth: int = 3, ) -> dict: prompt = ( f"Create a branching dialogue tree for NPC: {npc_name}\n" f"Topic: {topic}\n" f"Context: {context}\n" f"Create {depth} levels of branching. Include player choices that reveal " "different information. Some choices should have conditions " "(e.g., requires: high_reputation, has_item_X). " "Use node IDs like: node_start, node_1a, node_1b, node_end." ) result = await gemini_complete( prompt, thinking="high", json_schema=TREE_SCHEMA ) return json.loads(result["text"]) # The output is a complete dialogue tree you can import into # Dialogic (github.com/dialogic-godot/dialogic) or your own # custom dialogue system. Generate entire quest dialogue in seconds.
```

USE CASE 20.4

AI as a Dungeon Master — Reactive World Events

The AI watches the entire game state and spontaneously introduces events: a passing merchant, a bandit ambush, a rumour.

GDSCRIPT systems/dungeon_master.gd

```
extends Node ## The AI Dungeon Master watches the game world and spontaneously ## introduces events when the player seems bored, too successful, ## or in an area that has not been "touched" by the world recently. @export var check_interval_seconds: float = 120.0 var _timer: float = 0.0 func _process(delta: float) -> void: _timer += delta if _timer < check_interval_seconds: return _timer = 0.0 _check_world_state() func _check_world_state() -> void: var state := { "player_location": PlayerState.current_area, "time_in_area_min": PlayerState.time_in_current_area / 60, "last_event_min_ago": WorldState.last_event_time / 60, "player_gold": PlayerState.gold, "combat_last_5min": PlayerState.combats_last_5min, "player_mood_guess": PlayerState.estimated_mood, "active_quests": QuestLog.active_count(), } var prompt := ( "You are the Dungeon Master of a dark fantasy RPG. " "World state: " + JSON.stringify(state) + "\n" + "Should you introduce a world event right now? " + "Return JSON: {trigger: bool, event_type: string, " + "description: string, urgency: low|medium|high}\n" + "Event types: passing_npc, discovery, distant_sound, " + "environmental_change, minor_encounter, rumour_whisper" ) AIAgent.response_received.connect(_on_dm_decision, CONNECT_ONE_SHOT) AIAgent.ask(prompt, "none") func _on_dm_decision(text: String, _p: String) -> void: var s := text.find("{"); var e := text.rfind("}")+1 if s == -1: return var data: Dictionary = JSON.parse_string(text.substr(s, e-s)) if data.get("trigger", false): WorldEventSystem.trigger( data.get("event_type","passing_npc"), data.get("description",""), data.get("urgency","low"), )
```

USE CASE 20.5

AI Code Review Assistant Inside the Engine

Right-click any script in Godot's FileSystem dock. AI reviews it for bugs, performance issues, and GDScript anti-patterns.

GDSCRIPT addons/ai_agent/script_reviewer.gd

```
@tool extends EditorScript ## Run via Godot Editor menu: Script → Run ## Or attach a keyboard shortcut via EditorPlugin. func _run() -> void: # Get the currently open script in the editor var script := get_script() if not script: return var source := FileAccess.open(script.resource_path, FileAccess.READ) if not source: return var code := source.get_as_text() source.close() var prompt := ( "Review this Godot 4.6 GDScript for:\n" + "1. Logic bugs or null-pointer risks\n" + "2. Performance issues (in _process, unnecessary allocations)\n" + "3. Anti-patterns (wrong signal usage, not using await properly)\n" + "4. Missing error handling\n" + "Format: numbered list, specific line numbers where possible.\n\n" + "Script:\n```gdscript\n" + code + "\n```" ) AIAgent.response_received.connect( func(review, _p): _show_review(script.resource_path, review), CONNECT_ONE_SHOT) AIAgent.ask(prompt, "high", "gemini") func _show_review(path: String, review: String) -> void: print("\n=== AI Code Review: ", path, " ===") print(review) print("=====================================\n") # Or display in a custom editor dock panel
```

USE CASE 20.6

AI-Driven Cutscene Director

Describe what should happen narratively. AI generates camera positions, timing, and character animations.

PYTHON ai_bridge/cutscene_director.py

```
import json from provider_gemini import gemini_complete CUTSCENE_SCHEMA = { "type": "object",
"properties": { "shots": { "type": "array", "items": { "type": "object", "properties": { "duration_sec":
{"type":"number"},
"camera_position":{"type":"array","items":{"type":"number"},"minItems":3,"maxItems":3}, "camera_target":
{"type":"array","items":{"type":"number"},"minItems":3,"maxItems":3}, "character_anim":
{"type":"string"}, "character_path": {"type":"string"}, "dialogue": {"type":"string"}, "sfx":
{"type":"string"}, } } } } } async def direct_cutscene(narrative: str, available_anims: list[str]) ->
dict: prompt = ( f"Direct a cinematic cutscene for: {narrative}\n" f"Available character animations:
{available_anims}\n" "Generate camera shots with positions (x,y,z in Godot coordinate space), "
"character animations to play, and optional dialogue per shot. " "Make it cinematic -- vary shot
distance, use close-ups for emotional moments." ) result = await gemini_complete(prompt, thinking="high",
json_schema=CUTSCENE_SCHEMA) return json.loads(result["text"]) # Wire each shot to Godot's
AnimationPlayer, Camera3D, and AnimationTree. # The entire cutscene plays from AI output alone.
```

USE CASE 20.7

Persistent AI Memory Across Sessions

The NPC remembers what you told it last Tuesday. Not with a save file hack — with a proper vector-adjacent approach.

NPCs that remember specific player conversations across save sessions feel extraordinarily real. The trick is to summarise and compress important memories at session end and store them as text, then inject them into the system prompt next session:

PYTHON ai_bridge/memory_manager.py

```
import json, pathlib from provider_gemini import gemini_complete MEMORY_DIR =
pathlib.Path("saves/npc_memories") MEMORY_DIR.mkdir(parents=True, exist_ok=True) async def
compress_session(npc_id: str, conversation_log: list[str]) -> None: """Called at game save. Compresses
today's conversation into durable memories."""  prompt = ( "Summarise the most important things a
character should remember " "from this conversation log. Keep only facts that would change " "how they
behave in future interactions. Max 5 bullet points.\n\n" + "\n".join(conversation_log) ) result = await
gemini_complete(prompt, thinking="none") summary = result["text"] # Load existing memories, append new
summary mem_file = MEMORY_DIR / f"{npc_id}.txt" existing = mem_file.read_text() if mem_file.exists() else
"" # Keep only last 2000 chars of old memories to avoid bloat combined = (existing[-2000:] + "\n---\n" +
summary).strip() mem_file.write_text(combined) def load_memory(npc_id: str) -> str: """Inject into system
prompt at session start."""  mem_file = MEMORY_DIR / f"{npc_id}.txt" if mem_file.exists(): return
f"\n\nYour memories of this player:\n{mem_file.read_text()}" return "" # Result: NPCs that say "Last time
you were here you mentioned your sister. # Did you ever find her?" -- across completely separate play
sessions.
```

USE CASE 20.8

Procedural World Bible Expansion

Your game generates its own lore as players explore. Every discovered area adds to a living world document.

PYTHON ai_bridge/world_bible.py

```
import json, pathlib from provider_gemini import gemini_complete BIBLE_FILE =
pathlib.Path("saves/world_bible.json") def load_bible() -> dict: return
json.loads(BIBLE_FILE.read_text()) if BIBLE_FILE.exists() else {} def save_bible(bible: dict) -> None:
BIBLE_FILE.write_text(json.dumps(bible, indent=2)) async def expand_on_discovery(area_name: str,
visual_description: str) -> str: """ Player discovers a new area. AI writes its history into the world
bible. Returns flavour text for an in-game book or journal entry. """ bible = load_bible() existing_lore
= json.dumps(bible, indent=2)[:3000] # cap context prompt = ( f"Add to the world lore. Existing world
bible:\n{existing_lore}\n\n" f"The player just discovered: {area_name}\n" f"Visual description:
{visual_description}\n" "Write: (1) a 2-paragraph history for this area that fits the existing lore, "
"(2) a rumour locals might tell, (3) a short journal entry an explorer left here. " "Return JSON:
{history:str, rumour:str, journal_entry:str, " "new_lore_key:str, new_lore_value:str}" ) result = await
gemini_complete(prompt, thinking="medium") data = json.loads(result["text"]) # Expand the bible with the
new lore entry bible[data["new_lore_key"]] = data["new_lore_value"] save_bible(bible) return
data["journal_entry"] # show this in a Godot popup
```

USE CASE 20.9

AI Game Designer — Balance Suggestions from Player Data

Aggregate all player telemetry. Ask the AI what to change. It responds like a senior game designer.

PYTHON ai_bridge/game_designer.py

```
from provider_gemini import gemini_complete import json DESIGNER_SYSTEM = """ You are a senior game
designer with 15 years of experience in RPGs. You analyse player telemetry and give actionable, specific
design advice. Focus on: player retention, frustration points, pacing, and fun. Be direct. Give numbered
recommendations with specific changes. """ async def design_review(telemetry: dict) -> str: """ telemetry
example: { "avg_session_length_min": 22, "day_1_retention": 0.41, "top_quit_locations": ["tutorial_area",
"dungeon_level_3"], "most_died_to": "bat_swarm", "least_used_ability": "shield_bash",
"most_complained_about": "inventory is confusing", } """ prompt = ( "Review this game telemetry and give
me your top 5 design changes:\n" + json.dumps(telemetry, indent=2) ) result = await gemini_complete(
prompt, system=DESIGNER_SYSTEM, thinking="medium" ) return result["text"] # Run this weekly. Paste output
into your design doc. # A senior designer consultant costs $200/hour. # This costs $0.02 and available at
3am.
```

Complete File Reference

Every file you need for the complete stack, in dependency order:

File	§	Purpose
addons/ai_agent/plugin.cfg	§07	Plugin registration
addons/ai_agent/ai_agent_plugin.gd	§07	EditorPlugin — registers autoload
addons/ai_agent/ai_agent.gd	§07	Main autoload — AIAgent.ask()
addons/ai_agent/scene_builder.gd	§08	AI → node creation with rollback
addons/ai_agent/ai_streamer.gd	§11	Token-by-token streaming client
addons/ai_agent/env_patcher.gd	—	WorldEnvironment AI patching
ai_bridge/.env	§02	API keys — never commit
ai_bridge/provider_gemini.py	§03	Gemini 3 Flash client, vision, tools
ai_bridge/provider_glm.py	§04	GLM-5 client, Preserved Thinking
ai_bridge/offline_handler.py	§10	Ollama REST client
ai_bridge/provider_router.py	§05	Task-aware fallback chain
ai_bridge/bridge_server.py	§06	FastAPI server — main entry point
ai_bridge/glm_agent.py	§14	Stateful multi-turn GLM-5 agent
ai_bridge/cache_manager.py	§13	Explicit Gemini context caching
ai_bridge/retry.py	§12	Exponential backoff decorator
ai_bridge/json_utils.py	§12	JSON fence stripper and safe parser
ai_bridge/vlm_feedback.py	—	Screenshot → VLM → env patch loop
ai_bridge/tests/conftest.py	§15	Provider mock fixtures
ai_bridge/tests/test_router.py	§15	Router unit tests
.mcp.json	§09	Claude Code / Cursor MCP config
docker-compose.yml	§16	Local container stack
.github/workflows/ci.yml	§16	GitHub Actions CI pipeline

S 21

Performance Optimisation Deep Dive

Real latency numbers, token compression techniques, and strategies that cut costs 60-90% with zero quality loss.

| REAL NUMBERS FIRST

These are measured values for Godot AI workloads, not theoretical: Gemini 3 Flash with thinking=none: TTFT 0.6s, full 512-token reply 1.4s at 380 t/s. Gemini 3 Flash with thinking=medium: TTFT 5-7s, full 4096-token reply ~16s. GLM-5 via z.ai: TTFT 1.4s, throughput 63.5 t/s, full 4096 reply ~64s. GLM-5 via Cerebras: TTFT <0.5s, ~1000 t/s, full 4096 reply ~4s. Ollama glm4 on RTX 3090: TTFT ~1.5s, ~45 t/s, full 4096 reply ~91s. Bottom line: for real-time game use, streaming + thinking=none/minimal is the only viable path.

S21.1 Token compression -- 40-60% savings with no quality loss

The single highest-leverage optimisation. Godot .tscn scene files are extremely verbose. Compressing your scene context before sending it to the AI can cut input tokens by half.

PYTHON ai_bridge/token_utils.py

```
import re, tiktoken # Approximate token counter (Gemini uses different tokeniser but close enough) _enc =
tiktoken.get_encoding("cl100k_base") def count_tokens(text: str) -> int: return len(_enc.encode(text))
def compress_scene_context(raw: str) -> str: """ Typical savings: 45-55% fewer tokens on Godot .tscn
files. Run on your SCENE_CONTEXT string before every prompt. """ # 1. Strip XML/Godot comments out =
re.sub(r";.*", "", raw) # remove ; comments out = re.sub(r"\[node.*?]\s* ", lambda m: m.group().strip() +
" ", out) # 2. Collapse excess whitespace out = re.sub(r" {2,}", " ", out) out = re.sub(r" {3,}", " ",
out) # 3. Shorten verbose Godot class names (saves 2-5 tokens each) replacements = { "MeshInstance3D":
"MI3D", "CollisionShape3D": "CS3D", "CharacterBody3D": "CB3D", "DirectionalLight3D": "DL3D",
"WorldEnvironment": "WEnv", "NavigationAgent3D": "NavA", "AnimationPlayer": "AnimP",
"AudioStreamPlayer3D":"ASP3D", "GPUParticles3D": "GFX3D", "transform/position": "pos",
"transform/rotation": "rot", "transform/scale": "scl", } for long, short in replacements.items(): out =
out.replace(long, short) # 4. Remove UIDs (not needed by AI) out = re.sub(r"uid="uid://[a-z0-9]+"", "",
out) return out.strip() def trim_history(messages: list, max_tokens: int = 8000) -> list: """ Keep only
messages that fit within token budget. Always keeps: system message (index 0) + last N messages. """ if
not messages: return messages system = [messages[0]] rest = messages[1:] total =
count_tokens(messages[0].get("content","")) kept = [] for msg in reversed(rest): tokens =
count_tokens(str(msg.get("content",""))) if total + tokens > max_tokens: break kept.insert(0, msg) total
+= tokens return system + kept
```

S21.2 Request batching for non-real-time tasks

Not every AI call needs to be real-time. Batch tasks like overnight lore generation, asset description writing, and localisation at 50% of the standard price using Gemini's Batch API. Results arrive within 24 hours.

PYTHON ai_bridge/batch_jobs.py

```
from google import genai from google.genai import types import json, pathlib, os client = genai.Client(api_key=os.environ["GEMINI_API_KEY"]) def submit_batch(tasks: list[dict], job_name: str = "godot-batch") -> str: """ tasks format: [{"id": "task_1", "prompt": "...", "system": "..."}] Returns: job_name to poll later. Cost: 50% of standard pricing. Turnaround: up to 24 hours. Best for: lore generation, item descriptions, translation, screenshot audits. """ requests = [] for task in tasks: requests.append({ "request_id": task["id"], "model": "models/gemini-3-flash-preview", "contents": [{"parts": [{"text": task["prompt"]}]}], "system_instruction": {"parts": [{"text": task.get("system","")}]}, }) tmp = pathlib.Path("/tmp/batch_input.jsonl") tmp.write_text(" ".join(json.dumps(r) for r in requests)) uploaded = client.files.upload(file=str(tmp)) job = client.batches.create( model="models/gemini-3-flash-preview", src=uploaded.name, config=types.CreateBatchJobConfig(display_name=job_name), ) print(f"[Batch] Submitted {len(tasks)} tasks as job: {job.name}") return job.name def poll_batch(job_name: str) -> list[dict] | None: """Poll until complete. Returns None if still running."""  job = client.batches.get(name=job_name) if job.state.name not in ("JOB_STATE_SUCCEEDED", "JOB_STATE_FAILED"): print(f"[Batch] State: {job.state.name} -- still processing") return None if job.state.name == "JOB_STATE_FAILED": raise RuntimeError(f"Batch job failed: {job.error}") # Download results results = [] for line in client.files.download(job.dest).decode().splitlines(): results.append(json.loads(line)) return results
```

S21.3 Parallel racing for latency-critical NPC responses

PYTHON ai_bridge/race.py

```
import asyncio from provider_gemini import gemini_complete from provider_glm import glm_complete async def race_providers( prompt: str, timeout: float = 5.0, ) -> dict: """ Fire Gemini and GLM-5 simultaneously. Use whichever responds first. Cancel the loser. Cost: slightly higher (both providers bill for partial work). Benefit: guaranteed response within 5s regardless of which provider is slow. Best for: real-time NPC dialogue where latency > cost. """ tasks = { asyncio.create_task(gemini_complete(prompt, thinking="none"), name="gemini"): "gemini", asyncio.create_task(glm_complete(prompt, thinking=False), name="glm-5"): "glm-5", } try: done, pending = await asyncio.wait( tasks.keys(), return_when=asyncio.FIRST_COMPLETED, timeout=timeout, ) except asyncio.TimeoutError: for t in tasks: t.cancel() raise for t in pending: t.cancel() winner = next(iter(done)) return {"provider": tasks[winner], "result": winner.result()}
```

S21.4 Thinking level decision tree by task

Task type	thinking level	Reason	Approx TTFT
NPC one-liner response	none	No reasoning chain needed, pure generation	<1s
Item/location description	none	Creative output, not analytical	<1s
Quest objective text	minimal	Light coherence check sufficient	~1s
GDScript property setter	minimal	Simple, deterministic output	~1s
GDScript function generation	medium	Needs to plan logic before writing	~6s
Scene layout / node structure	medium	Spatial reasoning benefits from thinking	~6s
Script refactor / bug fix	high	Multi-step analysis of existing code	~15s

Task type	thinking level	Reason	Approx TTFT
Architecture / system design	high	Multiple competing options to evaluate	~15s
Economy balancing analysis	medium	Data interpretation, not deep code	~6s
Multi-step autonomous agent turn	medium (GLM-5 Preserved)	Agent uses memory, not just thinking	~8s

S 22

Cost Tracking & Budget Guardrails

Know exactly what you are spending. Hard caps, per-session limits, and real-time alerts.

PYTHON ai_bridge/cost_tracker.py

```
import time, json, pathlib, logging from collections import defaultdict log = logging.getLogger("costs") # Pricing per million tokens (March 2026) PRICES = { "gemini-3-flash-preview": {"input": 0.50, "output": 3.00, "cache": 0.05}, "glm-5": {"input": 1.00, "output": 3.20, "cache": 0.00}, "ollama": {"input": 0.00, "output": 0.00, "cache": 0.00}, } # In-memory ledger (persist to disk every 60s in production) _ledger: dict[str, list] = defaultdict(list) HOURLY_CAP_USD = float(10.00) # hard cap -- raise HTTPException if exceeded DAILY_CAP_USD = float(50.00) ALERT_THRESHOLD = float(0.80) # alert at 80% of cap def record(model: str, input_tokens: int, output_tokens: int, cached_tokens: int = 0) -> float: """Record a call and return its cost in USD."""" p = PRICES.get(model, PRICES["gemini-3-flash-preview"]) cost = ( (input_tokens - cached_tokens) * p["input"] / 1_000_000 + cached_tokens * p["cache"] / 1_000_000 + output_tokens * p["output"] / 1_000_000 ) _ledger["all"].append({"ts": time.time(), "model": model, "cost": cost}) _ledger[model].append({"ts": time.time(), "cost": cost}) _check_caps() return cost def spend_last(hours: float = 1.0) -> float: cutoff = time.time() - hours * 3600 return sum(e["cost"] for e in _ledger["all"] if e["ts"] > cutoff) def _check_caps() -> None: hourly = spend_last(1) daily = spend_last(24) if hourly >= HOURLY_CAP_USD: raise RuntimeError(f"Hourly AI budget exceeded: ${hourly:.4f}") if daily >= DAILY_CAP_USD: raise RuntimeError(f"Daily AI budget exceeded: ${daily:.4f}") if hourly >= HOURLY_CAP_USD * ALERT_THRESHOLD: log.warning(f"[costs] Hourly spend ${hourly:.4f} -- approaching cap") def report() -> dict: return { "hourly_usd": round(spend_last(1), 4), "daily_usd": round(spend_last(24), 4), "monthly_usd": round(spend_last(720), 2), "by_model": {m: round(sum(e["cost"] for e in v), 4) for m, v in _ledger.items() if m != "all"}, }
```

PYTHON ai_bridge/bridge_server.py -- add cost reporting endpoint

```
from cost_tracker import record, report as cost_report @app.get("/costs") async def costs(): """Live cost dashboard -- call from browser or Godot editor."""" return cost_report() # Wire into complete() so every call is tracked: # After: result = await route(req.prompt, ...) # Add: # if "result" in result: # r = result["result"] # if isinstance(r, dict) and "tokens" in r: # record(result["provider"], r.get("tokens",0), 0)
```

S 23

Production Deployment

Secrets management, health monitoring, graceful crash recovery, and scaling the bridge server.

| PRODUCTION CHECKLIST

[ ] API keys loaded from environment, NEVER from code or git. [ ] Bridge server runs behind a reverse proxy (nginx/caddy) with TLS. [ ] Rate limiting per IP (see bridge_server.py). [ ] Health endpoint monitored every 30s by uptime service. [ ] Cost alerts configured (see cost_tracker.py). [ ] Structured JSON logging with timestamps for every AI call. [ ] Circuit breakers tuned to your provider SLAs. [ ] Ollama running as offline fallback even in production. [ ] Automated restart on crash (systemd or Docker restart:unless-stopped). [ ] Secrets rotated every 90 days minimum.

S23.1 systemd service for the bridge server

INI /etc/systemd/system/godot-ai-bridge.service

```
[Unit] Description=Godot AI Bridge Server After=network.target Wants=ollama.service [Service] Type=simple User=gameserver WorkingDirectory=/opt/godot-ai/ai_bridge EnvironmentFile=/opt/godot-ai/ai_bridge/.env ExecStart=/opt/godot-ai/ai_bridge/.venv/bin/python bridge_server.py Restart=always RestartSec=5 StandardOutput=journal StandardError=journal [Install] WantedBy=multi-user.target # Enable and start: # sudo systemctl daemon-reload # sudo systemctl enable godot-ai-bridge # sudo systemctl start godot-ai-bridge # sudo journalctl -u godot-ai-bridge -f
```

S23.2 nginx reverse proxy with TLS

NGINX /etc/nginx/sites-available/godot-ai

```
server { listen 443 ssl; server_name ai.yourstudio.com; ssl_certificate /etc/letsencrypt/live/ai.yourstudio.com/fullchain.pem; ssl_certificate_key /etc/letsencrypt/live/ai.yourstudio.com/privkey.pem; # Only allow your game servers / dev machines allow 10.0.0.0/8; deny all; location / { proxy_pass http://127.0.0.1:8000; proxy_set_header Host $host; proxy_read_timeout 90s; # SSE support (streaming) proxy_buffering off; proxy_cache off; proxy_set_header X-Accel-Buffering no; } }
```

S23.3 Structured JSON logging for every AI call

PYTHON ai_bridge/logging_setup.py

```
import logging, json, time class JSONFormatter(logging.Formatter): def format(self, record: logging.LogRecord) -> str: entry = { "ts": round(time.time(), 3), "level": record.levelname, "logger": record.name, "msg": record.getMessage(), "provider": getattr(record, "provider", None), "tokens": getattr(record, "tokens", None), "cost_usd": getattr(record, "cost_usd", None), "latency": getattr(record, "latency", None), } return json.dumps({k: v for k, v in entry.items() if v is not None}) def setup_logging(level: str = "INFO") -> None: handler = logging.StreamHandler() handler.setFormatter(JSONFormatter()) root = logging.getLogger() root.setLevel(level) root.addHandler(handler) # Usage -- log a completed AI call with full context: def log_ai_call(provider: str, tokens: int, latency: float, cost: float) -> None: logging.getLogger("ai.call").info( "AI call complete", extra={"provider": provider, "tokens": tokens, "latency": latency, "cost_usd": cost} ) # Output: {"ts":1742040123.456,"level":"INFO","logger":"ai.call", # "msg":"AI call complete","provider":"gemini","tokens":1204, # "latency":6.2,"cost_usd":0.0024}
```

S 24

Multiplayer AI Considerations

Who controls AI in a multiplayer game? How do you handle latency, anti-cheat, and shared NPC state?

| THE FUNDAMENTAL RULE: AI IS SERVER AUTHORITY

In a multiplayer game, AI decisions must be authoritative on the server -- never on the client. A client that calls the AI bridge directly could fake responses to gain advantages. All AI calls go through your game server (or a dedicated AI server). Clients only receive the results, never the raw AI output.

S24.1 Architecture for multiplayer AI

Scenario	Wrong approach	Right approach
NPC dialogue in MMO	Each client calls bridge directly	Server calls bridge, multicasts result to nearby players
Dynamic difficulty	Client sends telemetry to bridge	Server aggregates all player telemetry, one bridge call
AI dungeon master	Client triggers world events	Dedicated AI server polls game state, pushes events to all
Quest generation	Each player generates own quests	Server generates quest pool, distributes to players
Voice command parsing	Client sends audio to bridge	Client transcribes locally, sends text to server, server calls bridge

S24.2 Shared NPC state across clients

PYTHON ai_bridge/npc_state_server.py

```
import asyncio, json, time from fastapi import WebSocket from provider_router import route #
Server-authoritative NPC state # All clients see the same NPC responses _npc_cache: dict[str, dict] = {}
_npc_lock = asyncio.Lock() async def get_npc_response( npc_id: str, player_input: str, room_id: str, ) ->
str: """ Returns the same NPC response to all players in the same room. Deduplicates: if two players
trigger the same NPC simultaneously, only one AI call is made and both get the same answer. """ cache_key
= f"{room_id}:{npc_id}:{hash(player_input)}" async with _npc_lock: if cache_key in _npc_cache: entry =
_npc_cache[cache_key] if time.time() - entry["ts"] < 30: # 30s cache TTL return entry["text"] # One AI
call for all clients result = await route(player_input, system=f"You are NPC {npc_id}.") text =
result.get("result", {}).get("text", "") async with _npc_lock: _npc_cache[cache_key] = {"text": text,
"ts": time.time()} return text # Multiplayer bridge endpoint: # POST /ai/npc with {npc_id, player_input,
room_id} # The Godot server calls this -- not the client directly.
```

S24.3 Anti-cheat -- AI response integrity

A malicious client could intercept the AI response and modify it before applying it to the game state. In competitive games, use HMAC signing on AI responses from the bridge:

PYTHON ai_bridge/signing.py

```
import hmac, hashlib, os, json SECRET = os.environ.get("AI_BRIDGE_HMAC_SECRET",
"change-this-in-production") def sign_response(data: dict) -> str: """Attach an HMAC signature the Godot
server can verify.""" payload = json.dumps(data, sort_keys=True).encode() sig = hmac.new(SECRET.encode(),
payload, hashlib.sha256).hexdigest() return sig def verify_response(data: dict, sig: str) -> bool: return
hmac.compare_digest(sign_response(data), sig) # Add to bridge_server.py complete() endpoint: #
result["sig"] = sign_response(result) # In Godot server-side GDScript: # func verify_ai(response:
Dictionary) -> bool: # # Send response + sig to auth endpoint for server-side verification # # Never trust
client to verify its own AI responses
```

S 25

Godot 4.6 Advanced AI Patterns

LibGodot + AI, IKModifier3D AI animation, SSR VLM tuning, ObjectDB snapshot AI debugging.

S25.1 LibGodot + AI: embedded AI-driven simulation

| GODOT 4.6 LIBGODOT

LibGodot (released with 4.6) lets you embed the Godot engine as a library inside your own Python or C++ application. This is directly relevant to AI integration: you can now run a Godot scene headlessly inside the same Python process as your bridge server, capture screenshots without a display server, and feed them to the VLM loop without any interprocess communication. Supported platforms: Linux, Windows, macOS.

PYTHON ai_bridge/libgodot_runner.py

```
""" Run Godot headlessly inside Python using LibGodot. Requires: Godot 4.6 libgodot.so / libgodot.dll
built as shared library. See: godotengine.org/releases/4.6 -- download LibGodot SDK. """ import ctypes,
pathlib # Load the Godot shared library godot_lib = ctypes.CDLL("/usr/local/lib/libgodot.so") def
capture_screenshot_headless( project_path: str, output_path: str = "/tmp/godot_frame.png", ) -> str: """
Launch Godot headlessly, run one frame, save screenshot. No display server required -- perfect for CI and
VLM loops. """ import subprocess result = subprocess.run([ "Godot_v4.6-stable_linux.x86_64",
"--headless", "--path", project_path, "--script", "res://addons/ai_agent/headless_screenshot.gd",
"--quit-after", "1", ], capture_output=True, timeout=30) if result.returncode != 0: raise
RuntimeError(f"Godot headless failed: {result.stderr.decode()}") return output_path
```

S25.2 AI-driven IKModifier3D animation

| GODOT 4.6 IKMODIFIER3D

IKModifier3D returns in Godot 4.6 with a proper node-based constraint system. AI can now procedurally control IK targets -- making an NPC's eyes follow the player, hands reach for specific objects, or body pose adapt to uneven terrain -- without hand-authored animation clips for every scenario.

GDSCRIPT characters/ai_ik_controller.gd

```
extends Node ## AI-driven IK target positioning. ## Ask the AI where limbs should move, apply via
IKModifier3D targets. @onready var ik_head: IKModifier3D = $Skeleton3D/IKHead @onready var ik_r_hand:
IKModifier3D = $Skeleton3D/IKRightHand @onready var ik_l_hand: IKModifier3D = $Skeleton3D/IKLeftHand func
aim_at_point_of_interest(world_pos: Vector3) -> void: """AI identifies what the character should look
at."""  var prompt := ( "Character at %s, point of interest at %s. " % [global_position, world_pos] +
"Return JSON: {head_target: [x,y,z], emotion: neutral|curious|alert}" )
AIAgent.response_received.connect(_on_ik_response, CONNECT_ONE_SHOT) AIAgent.ask(prompt, "none") # Fast,
no thinking needed func _on_ik_response(text: String, _p: String) -> void: var s := text.find("{"); var e
:= text.rfind("}")+1 if s == -1: return var data: Dictionary = JSON.parse_string(text.substr(s,e-s)) var
target_arr = data.get("head_target", []) if target_arr.size() == 3: ik_head.target_position =
Vector3(target_arr[0],target_arr[1],target_arr[2]) # Also apply emotion via AnimationTree parameters
$AnimationTree.set("parameters/emotion/blend_position",
["neutral","curious","alert"].find(data.get("emotion","neutral")) * 0.5)
```

S25.3 AI-assisted ObjectDB snapshot debugging (Godot 4.6)

| GODOT 4.6 OBJECTDB SNAPSHOTS

Godot 4.6 adds ObjectDB snapshot and diffing in the debugger. You can capture the full state of all live objects, then compare two snapshots to find memory leaks, orphaned nodes, and unintended object growth. Feed these diffs to the AI for intelligent diagnosis.

PYTHON ai_bridge/objectdb_analyser.py

```
from provider_gemini import gemini_complete import json async def analyse_objectdb_diff(snapshot_before: dict, snapshot_after: dict) -> str: """ Feed two ObjectDB snapshots to the AI. It identifies memory leaks, orphaned nodes, and unexpected growth. """ diff = { "new_objects": [ obj for obj in snapshot_after["objects"] if obj["id"] not in {o["id"] for o in snapshot_before["objects"]} ], "deleted_objects": [ obj for obj in snapshot_before["objects"] if obj["id"] not in {o["id"] for o in snapshot_after["objects"]} ], "count_before": len(snapshot_before["objects"]), "count_after": len(snapshot_after["objects"]), } prompt = ( "Analyse this Godot ObjectDB snapshot diff for memory leaks:\n" + json.dumps(diff, indent=2)[:4000] + "\nIdentify: (1) nodes not freed after scene change, " + "(2) textures/resources held longer than expected, " + "(3) any pattern suggesting a leak. Be specific with node names." ) result = await gemini_complete(prompt, thinking="high") return result["text"]
```

S 26

More Secret Creative Use Cases (21-30)

Ten more applications most developers have never considered. Each one is genuinely unexplored territory.

USE CASE 21.1

AI-Powered Accessibility System

AI describes what is happening on screen for visually impaired players, in real time.

Most games have zero meaningful accessibility for visually impaired players. A VLM running on screenshots can narrate what is on screen, describe enemy positions, warn about incoming danger, and read UI elements aloud. This costs $0.006 per screenshot and can be triggered by a keyboard shortcut or run continuously every 3 seconds:

PYTHON ai_bridge/accessibility.py

```
from provider_gemini import gemini_complete async def describe_screen( screenshot_path: str, context: str = "", ) -> str: """ Describe what is happening on screen for a visually impaired player. Returns a concise spoken description suitable for screen reader TTS. """ prompt = ( "Describe this game screen for a visually impaired player. " "Be concise (2-3 sentences). Include: " "player position/health, nearby threats, important UI state, " "and any action required. Avoid jargon -- speak naturally. " + (f"Context: {context}" if context else "") ) result = await gemini_complete(prompt, thinking="none", image_path=screenshot_path) return result["text"] # Wire to Godot TTS: # DisplayServer.tts_speak(description, DisplayServer.tts_get_voices()[0]) # Cost: $0.006 per call. At 1 call/3s: ~$0.007/minute. $0.43/hour. # This is the entire cost of making your game playable by blind players.
```

USE CASE 21.2

Procedural Texture Description Pipeline

Describe a texture in words. AI generates the description that drives your procedural texture system.

PYTHON ai_bridge/texture_pipeline.py

```
from provider_gemini import gemini_complete import json TEXTURE_SCHEMA = { "type": "object", "properties": { "base_colour": {"type": "string", "description": "hex colour"}, "roughness": {"type": "number", "minimum": 0, "maximum": 1}, "metallic": {"type": "number", "minimum": 0, "maximum": 1}, "noise_scale": {"type": "number"}, "noise_strength":{"type":"number","minimum":0,"maximum":1}, "detail_type": {"type": "string", "enum": ["cracks","veins","spots","grain","smooth"]}, "emission_colour":{"type":"string","description":"hex or none"}, "normal_intensity":{"type":"number","minimum":0,"maximum":2}, } } async def describe_to_texture(description: str) -> dict: """ Input: "ancient cracked stone, dark grey with faint orange moss", Output: shader parameters you can feed directly to your procedural texture shader. """ prompt = ( f"Generate PBR texture parameters for: {description}\n" "Return physically plausible values for a game material." ) result = await gemini_complete(prompt, thinking="minimal", json_schema=TEXTURE_SCHEMA) return json.loads(result["text"]) # In Godot: apply the dict as ShaderMaterial parameters. # Build a library of 500 textures overnight with a batch job at 50% cost.
```

USE CASE 21.3

AI-Driven Tutorial System That Watches the Player

The tutorial only explains what the player has NOT figured out. Inferred from play, not from a checklist.

GDSCRIPT systems/smart_tutorial.gd

```
extends Node ## Watches player actions and teaches only what they seem to not know. ## Traditional tutorials: fixed checklist. ## This: adaptive, never explains what the player already knows. var _actions_observed: Dictionary = {} var _tutorial_shown: Array = [] func record_action(action: String) -> void: _actions_observed[action] = _actions_observed.get(action, 0) + 1 func check_for_needed_hint(current_challenge: String) -> void: var prompt := ( "Player is facing: " + current_challenge + "\n" + "Actions they have performed: " + JSON.stringify(_actions_observed) + "\n" + "Hints already shown: " + JSON.stringify(_tutorial_shown) + "\n" + "Should we show a hint? Return JSON: " + "{show: bool, hint: string, mechanic_being_taught: string}" + " Only suggest a hint if the player has been stuck > 2 minutes " + "or is clearly missing a core mechanic." ) AIAgent.response_received.connect(_on_hint_decision, CONNECT_ONE_SHOT) AIAgent.ask(prompt, "none") func _on_hint_decision(text: String, _p: String) -> void: var s := text.find("{"); var e := text.rfind("}")+1 if s == -1: return var data: Dictionary = JSON.parse_string(text.substr(s,e-s)) if data.get("show", false): var hint: String = data.get("hint", "") _tutorial_shown.append(data.get("mechanic_being_taught","")) HUDController.show_hint(hint)
```

USE CASE 21.4

Community Modding Assistant

Your modders talk to an AI that knows your entire engine API. It writes GDScript mods for them.

Lower the barrier to modding by giving your community an AI assistant that knows your game API, your custom nodes, and your scripting conventions. A modder who cannot code can describe what they want and get working GDScript:

PYTHON ai_bridge/modding_assistant.py

```
from provider_gemini import gemini_complete YOUR_GAME_API = """ Custom nodes available to modders: QuestNode: add_objective(text), complete(), fail() ItemNode: give_to_player(), drop_at(pos), set_value(n) SpawnPoint: spawn(enemy_type, count), set_interval(s) Signals on GameEvents autoload: player_died, quest_completed(quest_id), item_collected(item_id) Modding conventions: Place mods in res://mods//mod.gd Extend ModBase and implement _on_load() and _on_unload() """ MOD_SYSTEM = ( "You are a modding assistant for [Game Name]. " "You help non-programmers create mods using GDScript 2.0. " "Always produce complete, working mod files. " "Use only the APIs listed below.\n\n" + YOUR_GAME_API ) async def generate_mod(description: str) -> str: prompt = ( f"Create a Godot 4.6 mod file for this request: {description}\n" "Return only the complete .gd file, no explanation." ) result = await gemini_complete(prompt, system=MOD_SYSTEM, thinking="medium") return result["text"] # Host this behind a simple web form. # Your modders describe what they want in plain English. # They download a working .gd file instantly.
```

USE CASE 21.5

AI-Generated Loading Screen Lore

Every loading screen shows a piece of world lore generated specifically for what the player just did.

GDSCRIPT ui/smart_loading_screen.gd

```
extends CanvasLayer ## Shows AI-generated lore relevant to the current game moment. ## Runs during actual scene transitions -- the loading time is free AI time. @onready var lore_label: RichTextLabel = $LoreLabel @onready var hint_label: Label = $HintLabel var _last_location: String = "" var _player_actions: Array = [] func show_for_transition(from_scene: String, to_scene: String) -> void: lore_label.text = "..." show() var prompt := ( "Player is loading from %s to %s. " % [from_scene, to_scene] + "Recent actions: " + JSON.stringify(_player_actions.slice(-5)) + "\n" + "Write: (1) one intriguing lore fact about the destination (2 sentences), " + "(2) one subtle gameplay hint relevant to what the player just did. " + "Return JSON: {lore: string, hint: string}" ) AIAgent.response_received.connect(_on_lore, CONNECT_ONE_SHOT) AIAgent.ask(prompt, "none") # Fast -- minimal thinking func _on_lore(text: String, _p: String) -> void: var s := text.find("{"); var e := text.rfind("}")+1 if s == -1: return var data: Dictionary = JSON.parse_string(text.substr(s,e-s)) lore_label.text = data.get("lore", "") hint_label.text = data.get("hint", "")
```

USE CASE 21.6

AI Play Data --> Better AI

Your game generates training signal for the AI by recording which AI responses players actually liked.

This is genuinely unexplored: use your players as a preference data collection system. Track which AI-generated NPC lines players re-read, which quests they complete, which difficulty adjustments they accept. Build a preference dataset. Use it to write better system prompts.

PYTHON ai_bridge/preference_collector.py

```
import json, pathlib, time PREF_FILE = pathlib.Path("data/ai_preferences.jsonl") def record_preference( prompt: str, response: str, signal: str, # "liked", "skipped", "replayed", "ignored" context: dict, ) -> None: """ Record implicit feedback on AI responses. signal sources: "liked" -- player pressed a "remember this" button "replayed" -- player opened dialogue history "skipped" -- player mashed through text quickly "ignored" -- NPC dialogue triggered but player walked away """ entry = { "ts": time.time(), "prompt": prompt[:500], "response":response[:500], "signal": signal, "context": context, } with PREF_FILE.open("a") as f: f.write(json.dumps(entry) + "\n") async def analyse_preferences() -> str: """Weekly: feed preference data to AI, get better system prompts."""  from provider_gemini import gemini_complete entries = [json.loads(l) for l in PREF_FILE.read_text().splitlines()[-200:]] liked = [e for e in entries if e["signal"] in ("liked","replayed")] skipped = [e for e in entries if e["signal"] in ("skipped","ignored")] prompt = ( f"Players liked these {len(liked)} AI responses and " f"skipped/ignored these {len(skipped)}.\n" "Analyse what makes the liked ones better. " "Suggest a revised system prompt that produces more of the liked style." ) result = await gemini_complete(prompt, thinking="high") return result["text"]
```

USE CASE 21.7

AI Environment Artist -- Biome-Aware Prop Placement

AI reads biome data and places props (rocks, plants, debris) that make environmental sense.

GDSCRIPT world/ai_prop_placer.gd

```
extends Node3D ## AI-driven prop placement based on biome, elevation, and proximity data. ## Produces organic-looking environments without hand-placing every asset. @export var biome: String = "forest" @export var area_size: Vector2 = Vector2(50, 50) @export var prop_budget: int = 30 var _prop_library := { "forest": ["tree_oak","tree_pine","bush_fern","rock_moss","log_fallen"], "desert": ["cactus_tall","cactus_squat","rock_sandstone","dead_tree","tumbleweed"], "dungeon": ["torch_wall","barrel","crate","bones","stalagmite","puddle"], } func populate() -> void: var available := _prop_library.get(biome, []) var prompt := ( "Place %d props in a %s area of %s meters. " % [prop_budget, biome, area_size] + "Available props: " + JSON.stringify(available) + "\n" + "Place them naturally -- cluster plants near water, " + "scatter rocks, make groups look organic not grid-like. " + "Return JSON array: [{prop: string, x: float, z: float, rot_y: float, scale: float}]" ) AIAgent.response_received.connect(_on_placement, CONNECT_ONE_SHOT) AIAgent.ask(prompt, "medium") func _on_placement(text: String, _p: String) -> void: var s := text.find("["); var e := text.rfind("]")+1 if s == -1: return var placements = JSON.parse_string(text.substr(s,e-s)) if not placements is Array: return for p in placements: var scene := load("res://assets/props/" + p.prop + ".tscn") as PackedScene if not scene: continue var node := scene.instantiate() node.position = Vector3(p.x, 0, p.z) node.rotation.y = deg_to_rad(p.get("rot_y", 0.0)) node.scale = Vector3.ONE * p.get("scale", 1.0) add_child(node)
```

USE CASE 21.8

AI Speedrun Coach

AI watches a speedrun attempt in real time and coaches the player on optimal routes and techniques.

PYTHON ai_bridge/speedrun_coach.py

```
from provider_gemini import gemini_complete import json # You provide the world record splits and known optimal routes WR_DATA = { "route_name": "any_percent_glitchless", "splits": [ {"name": "Forest Exit", "wr_time_s": 142, "notes": "Skip the troll via west path"}, {"name": "Castle Entry", "wr_time_s": 318, "notes": "Jump cancel on drawbridge"}, {"name": "Boss 1 Kill", "wr_time_s": 445, "notes": "Phase 2 stagger exploit"}, ] } async def coach_split( split_name: str, player_time: float, player_deaths: int, screenshot: str, ) -> str: wr_split = next((s for s in WR_DATA["splits"] if s["name"] == split_name), None) if not wr_split: return "" delta = player_time - wr_split["wr_time_s"] prompt = ( f"Speedrun coaching for split: {split_name}\n" f"Player time: {player_time:.1f}s | World record: {wr_split['wr_time_s']}s | " f"Difference: {delta:+.1f}s\n" f"Deaths: {player_deaths}\n" f"WR notes: {wr_split['notes']}\n" "Give one specific, actionable coaching tip based on the screenshot. " "Keep it under 2 sentences. Be encouraging but precise." ) result = await gemini_complete(prompt, thinking="none", image_path=screenshot) return result["text"]
```

USE CASE 21.9

AI-Powered Narrative Consistency Checker

Before publishing a patch: AI scans all dialogue, lore entries, and quest text for contradictions.

PYTHON tools/narrative_checker.py

```
import pathlib, asyncio from provider_gemini import gemini_complete async def
check_consistency(content_dir: str) -> list[dict]: """ Scan all .txt/.json content files for narrative
contradictions. Returns a list of {file, issue, severity} dicts. Run before every content patch ship. """
content_dir_p = pathlib.Path(content_dir) all_text = "" file_map = {} for f in
content_dir_p.rglob("*.txt"): text = f.read_text()[:2000] file_map[f.name] = text all_text += f"\n\n===
{f.name} ===\n{text}" prompt = ( "You are a narrative editor. Review this game content for:\n" "1.
Factual contradictions (NPC says X in one file, not-X in another)\n" "2. Timeline inconsistencies\n" "3.
Character voice inconsistencies (same NPC sounds different)\n" "4. Lore conflicts\n\n" "Content:\n" +
all_text[:30000] ) result = await gemini_complete(prompt, thinking="high") return result["text"] if
__name__ == "__main__": findings = asyncio.run(check_consistency("res://data/dialogue")) print(findings)
# Run weekly before patch day. Catches the "Mira died in Act 1 but # appears in Act 3" type of error
automatically.
```

USE CASE 21.10

AI as a Second Pair of Eyes for Accessibility Auditing

Automatically audit your UI for contrast ratios, text size, and colour-blindness issues.

PYTHON tools/accessibility_audit.py

```
from provider_gemini import gemini_complete import pathlib AUDIT_PROMPT = """ You are an accessibility
auditor for video games. Analyse this UI screenshot and check for: 1. COLOUR CONTRAST: Do text and
backgrounds meet WCAG 2.1 AA (4.5:1 ratio)? 2. TEXT SIZE: Is any text smaller than 16px equivalent? 3.
COLOUR ONLY INFO: Is any critical info conveyed only by colour (no shape/text)? 4. COLOUR BLINDNESS:
Would this UI be unreadable for deuteranopia (red/green)? 5. MOTION: Any flashing elements that could
trigger photosensitive epilepsy? 6. BUTTON SIZE: Are interactive elements at least 44x44px equivalent?
Return JSON: { "issues": [{"type": str, "severity": "critical|major|minor", "description": str}],
"passed": ["list of checks that passed"], "accessibility_score": int 0-100 } """ async def
audit_screenshot(screenshot_path: str) -> dict: import json result = await gemini_complete(AUDIT_PROMPT,
thinking="medium", image_path=screenshot_path) raw = result["text"] s, e = raw.find("{"),
raw.rfind("}")+1 if s == -1: return {"error": "No JSON in response"} return json.loads(raw[s:e]) # Run on
every screen in your game before each release. # Cost: $0.006 per screenshot. A 50-screen audit costs
$0.30. # Catches issues that would otherwise require a professional accessibility audit.
```

APP

Complete Reference Appendix

Every endpoint, every environment variable, every error code, and a full glossary.

A1 All Bridge Server Endpoints

Method	Endpoint	Body / Params	Returns	Used In
GET	/health	none	{"ok":true,"version":"1.0.0"}	Health monitoring
GET	/providers	none	{"gemini":bool,"glm-5":bool,"ollama":bool}	Provider status
GET	/metrics	none	{"circuit_fails":{...}}	Observability
GET	/costs	none	{"hourly_usd":...,"daily_usd":...}	Cost dashboard
POST	/ai/complete	AiReq	{"provider":str,"result":{...}}	Main AI call
POST	/ai/stream	AiReq	SSE text/event-stream	Streaming tokens
POST	/ai/agent	AgentReq	{"actions":[...],"text":str}	Multi-turn agent
POST	/env/patch	{"patch":{...}}	{"status":"queued","patch":{...}}	VLM env tuning
POST	/scene/screenshot	none	{"status":"screenshot_requested"}	VLM loop trigger
POST	/batch/submit	[tasks]	{"job_name":str}	Batch API jobs
GET	/batch/poll	?job=name	[results] or null	Batch status
GET	/docs	none	Swagger UI	Interactive API docs

A2 All Environment Variables

Variable	Default	Required	Description
GEMINI_API_KEY	none	Yes (cloud)	Google AI Studio key

Variable	Default	Required	Description
ZAI_API_KEY	none	Yes (cloud)	z.ai key for GLM-5
ZAI_BASE_URL	https://api.z.ai/api/paas/v4	No	Override for local GLM-5 via vLLM
OPENROUTER_API_KEY	none	No	OpenRouter for cheaper GLM-5
USE_OPENROUTER	false	No	Route GLM-5 calls through OpenRouter
OLLAMA_URL	http://localhost:11434	No	Ollama server base URL
OLLAMA_MODEL	glm4	No	Ollama model tag
AI_OFFLINE	false	No	true = skip all cloud providers
BRIDGE_HOST	127.0.0.1	No	Bind address for FastAPI server
BRIDGE_PORT	8000	No	Port for FastAPI server
LOG_LEVEL	INFO	No	DEBUG|INFO|WARNING|ERROR
AI_BRIDGE_HMAC_SECRET	change-me	No	HMAC key for response signing
HOURLY_CAP_USD	10.00	No	Hard hourly spend cap
DAILY_CAP_USD	50.00	No	Hard daily spend cap

A3 Common Error Codes Reference

Code	Source	Meaning	Fix
429	Any cloud provider	Rate limit exceeded	Exponential backoff -- see retry.py
400 context	Gemini / GLM-5	Prompt exceeds context window	Trim with trim_history() in token_utils.py
400 filter	Gemini / GLM-5	Content policy block	Rephrase prompt, avoid flagged topics
401	Any cloud provider	API key invalid or expired	Check .env, rotate key
503	Any cloud provider	Provider temporarily down	Router falls to next provider automatically
All failed	Router	All three providers unavailable	Check /providers endpoint, verify keys and Ollama
Bad JSON	Bridge	AI returned non-JSON when JSON expected	Use safe_json() from json_utils.py
HTTP 0	Godot HTTPRequest	Bridge server not running	Start python bridge_server.py first

Code	Source	Meaning	Fix
Timeout (Godot)	Godot HTTPRequest	Request exceeded 90s TIMEOUT	Reduce thinking level or use streaming
HMAC mismatch	Signing module	Response tampered (multiplayer)	Investigate -- potential cheat attempt

A4 All GitHub Repositories Referenced in This Guide

MCP SERVERS	
`github.com/satelliteoflove/godot-mcp`	Recommended MCP v2.11.0 -- npx, runtime control, screenshots, Godot 4.6 verified
`github.com/Coding-Solo/godot-mcp`	32 tools, UID handling, launch editor, run project, MIT
`github.com/tomyud1/godot-mcp`	32 tools + browser visualiser at :6510, AssetLib install
`github.com/ee0pdt/Godot-MCP`	Full script + scene read/write, stdio transport
`github.com/bradypp/godot-mcp`	Auto Godot detection, read-only safe mode for CI
`github.com/ryanmazzolini/minimal-godot-mcp`	LSP + DAP diagnostics, no addon required
`github.com/briancain/GodotServer-Docker`	Docker container for headless Godot 4.x server deployment
AI MODELS & WEIGHTS	
`huggingface.co/zai-org/GLM-5`	GLM-5 weights -- MIT, 744B/40B MoE, BF16 and FP8
`huggingface.co/zai-org/GLM-5-FP8`	FP8 quantised for vLLM/SGLang -- 8x A100 minimum
`huggingface.co/zai-org/GLM-4.7`	GLM-4.7 weights for full model local deployment
TOOLS & RUNTIMES	
`ollama.com`	Ollama -- local model server with OpenAI-compatible REST API
`github.com/vllm-project/vllm`	vLLM -- high-throughput inference, supports GLM-5 FP8
`github.com/sgl-project/sglang`	SGLang -- faster than vLLM on Hopper/Blackwell via EAGLE
API & DOCUMENTATION	
`aistudio.google.com`	Google AI Studio -- Gemini API keys and usage dashboard
`ai.google.dev/gemini-api/docs`	Gemini API reference, model list, pricing, feature flags
`docs.z.ai/guides/llm/glm-5`	GLM-5 API reference including thinking mode parameters
`openrouter.ai/models/zai-org/glm-5`	OpenRouter GLM-5 -- 28% cheaper than z.ai direct
`artificialanalysis.ai/models/glm-5`	Independent GLM-5 benchmarks: latency, throughput, quality
`godotengine.org/releases/4.6`	Official Godot 4.6 release notes and LibGodot SDK

A5 Glossary

Term	Definition
MCP	Model Context Protocol -- JSON-RPC 2.0 protocol connecting AI assistants to external tools like the Godot editor
SSE	Server-Sent Events -- HTTP streaming protocol used to push AI tokens to GDScript in real time
VLM	Vision Language Model -- AI that accepts both images and text; used for screenshot analysis
Preserved Thinking	GLM-5 feature: reasoning chain retained across ALL turns in a session; prevents forgetting prior decisions
thinking_level	Gemini 3 Flash parameter: none/minimal/medium/high -- controls how many reasoning tokens the model generates before responding
Circuit Breaker	Pattern that stops routing to a provider after N consecutive failures, preventing cascading errors
Implicit Caching	Gemini auto-caching: identical prompt prefixes over 2048 tokens are cached at 90% discount automatically
Explicit Caching	Manual Gemini cache: upload a context block, pay $1/M/hr storage, guaranteed cache hits
TTFT	Time To First Token -- latency from request sent to first character received
Jolt Physics	Godot 4.6 default 3D physics engine -- deterministic, stable; replaces Bullet/GodotPhysics
LibGodot	Godot 4.6 feature: embed Godot as a shared library in your own Python/C++ application
Unique Node IDs	Godot 4.6 feature: stable per-node ID via get_instance_id() that survives rename and reparent
IKModifier3D	Godot 4.6 node: applies inverse kinematics to Skeleton3D bones at runtime
FP8	Float8 quantisation: reduces GLM-5 VRAM requirement from ~600GB to ~240GB on H100 GPUs
SWE-bench	Software Engineering Benchmark: measures AI ability to solve real GitHub issues; gold standard for coding quality
Provider Router	bridge_server component: selects which AI provider to use based on task type, availability, and circuit state
Autoload	Godot singleton: a node accessible from any script in the project without get_node() -- AIAgent is an Autoload
ObjectDB Snapshot	Godot 4.6 debugger feature: capture all live object states and diff two snapshots to find memory leaks

Godot 4.6 AI Integration Guide -- Three-Part Complete Edition -- March 15, 2026. Part 1: Architecture, Setup, Gemini 3 Flash, GLM-5, Router, Bridge Server, Godot Plugin, Scene Builder, MCP, Offline. Part 2: Streaming, Error Handling, Caching, Multi-Turn Agents, Testing, CI/CD, Model Reference, Standard Use Cases, Advanced Use Cases, Secret Use Cases (1-10). Part 3: Performance, Cost Tracking, Production, Multiplayer AI, Godot 4.6 Advanced, Secret Use Cases (11-20), Reference Appendix. Sources: godotengine.org/releases/4.6 -- aistudio.google.com -- docs.z.ai -- huggingface.co/zai-org/GLM-5 -- artificialanalysis.ai -- github.com/satelliteoflove/godot-mcp